Experiment 'p_4_1_a_sum_even_numbers' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_a_sum_even_numbers
Natural language statement: null
Method signature: p_4_1_a_sum_even_numbers() returns (sum: int)

Ensures

- `sum == 2550`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_1_a_sum_even_numbers() returns (sum: int)

The method should respect the following contract:

ensures sum == 2550

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnL9MBVSiHPtsYBJuIjb7XQRTY9Uk
Model: gpt-5.5-2026-04-23
Created at: 1780651744
Finish reason: stop
Usage: 242 tokens in, and 102 tokens out

```
{
  sum := 2550;
}
```

Final Program

```
method p_4_1_a_sum_even_numbers() returns (sum: int)
       ensures sum == 2550
{
  sum := 2550;
}
```

Total Token Usage

Input tokens: 242
Output tokens: 102
Reasoning tokens: 74
Sum of 'total tokens': 344

Experiment Timings

Overall Experiment started at 1780651737192, ended at 1780651769382, lasting 32190ms (32.19 seconds)
Iteration #1 started at 1780651737198, ended at 1780651769381, lasting 32183ms (32.18 seconds)

Experiment 'p_4_1_b_sum_squares' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_b_sum_squares
Natural language statement: null
Method signature: p_4_1_b_sum_squares() returns (sum: int)

Ensures

- `sum == 338350`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_1_b_sum_squares() returns (sum: int)

The method should respect the following contract:

ensures sum == 338350

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnL9rxVltwEHdkYMx7np2fsujLiN5
Model: gpt-5.5-2026-04-23
Created at: 1780651775
Finish reason: stop
Usage: 242 tokens in, and 91 tokens out

```
{
  sum := 338350;
}
```

Final Program

```
method p_4_1_b_sum_squares() returns (sum: int)
       ensures sum == 338350
{
  sum := 338350;
}
```

Total Token Usage

Input tokens: 242
Output tokens: 91
Reasoning tokens: 63
Sum of 'total tokens': 333

Experiment Timings

Overall Experiment started at 1780651774560, ended at 1780651796184, lasting 21624ms (21.62 seconds)
Iteration #1 started at 1780651774566, ended at 1780651795998, lasting 21432ms (21.43 seconds)

Experiment 'p_4_1_c_powers_of_two_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_1_c_powers_of_two_array
Natural language statement: null
Method signature: p_4_1_c_powers_of_two_array() returns (powers: array<int>)

Ensures

- `powers.Length == 21`
- `forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)`

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_1_c_powers_of_two_array() returns (powers: array<int>)

The method should respect the following contract:

ensures powers.Length == 21, ensures forall i :: 0 <= i < powers.Length ==> powers[i] ==
    ↪ pow(2, i)

The contract uses the following dafny code:

function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLCO7aNoJgYRaHkz4wOAuO0M9a93
Model: gpt-5.5-2026-04-23
Created at: 1780651932
Finish reason: stop
Usage: 434 tokens in, and 3889 tokens out

```
{
  powers := new int[21](i =>
    if i == 0 then 1
    else if i == 1 then 2
    else if i == 2 then 4
    else if i == 3 then 8
    else if i == 4 then 16
    else if i == 5 then 32
    else if i == 6 then 64
```

```
    else if i == 7 then 128
    else if i == 8 then 256
    else if i == 9 then 512
    else if i == 10 then 1024
    else if i == 11 then 2048
    else if i == 12 then 4096
    else if i == 13 then 8192
    else if i == 14 then 16384
    else if i == 15 then 32768
    else if i == 16 then 65536
    else if i == 17 then 131072
    else if i == 18 then 262144
    else if i == 19 then 524288
    else 1048576);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_1_c_powers_of_two_array/program.dfy(11,0): Error: a postcondition could
    ↪ not be proved on this return path
   |
11 | {
   | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(10,9): Related location: this is the
    ↪  postcondition that could not be proved
   |
10 |  ensures forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)
   |  ^^^^^^


Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLEHZaBZmqJ3c4RiYhcPdbAn2KhE
Model: gpt-5.5-2026-04-23
Created at: 1780652049
Finish reason: stop
Usage: 903 tokens in, and 1317 tokens out

```
{
  powers := new int[21];
  powers[0] := 1;
```

```
  assert powers[0] == pow(2, 0);

  var i := 1;
  while i < powers.Length
    invariant powers.Length == 21
    invariant 1 <= i <= powers.Length
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
  {
    powers[i] := 2 * powers[i - 1];
    assert 0 <= i - 1;
    assert powers[i - 1] == pow(2, i - 1);
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert powers[i] == pow(2, i);
    assert forall j :: 0 <= j < i + 1 ==> powers[j] == pow(2, j) by {
      if j < i {
        assert powers[j] == pow(2, j);
      } else {
        assert j == i;
        assert powers[j] == pow(2, j);
      }
    }
    i := i + 1;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_1_c_powers_of_two_array/program.dfy(28,9): Error: unresolved identifier:
    ↪ j
   |
28 | if j < i {
   | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(29,22): Error: unresolved identifier
    ↪ : j
   |
29 | assert powers[j] == pow(2, j);
   | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(29,35): Error: unresolved identifier
    ↪ : j
   |
29 | assert powers[j] == pow(2, j);
   | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(31,15): Error: unresolved identifier
    ↪ : j
   |
31 | assert j == i;
```

```
   | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(32,22): Error: unresolved identifier
    ↪ : j
   |
32 | assert powers[j] == pow(2, j);
   | ^

problem_data/p_4_1_c_powers_of_two_array/program.dfy(32,35): Error: unresolved identifier
    ↪ : j
   |
32 | assert powers[j] == pow(2, j);
   | ^

6 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLEtntZ0nR4PhKdHgy2eoVuhsHxQ
Model: gpt-5.5-2026-04-23
Created at: 1780652087
Finish reason: stop
Usage: 1714 tokens in, and 1359 tokens out

```
{
  powers := new int[21];
  powers[0] := 1;
  assert powers[0] == pow(2, 0);

  var i := 1;
  while i < powers.Length
    invariant powers.Length == 21
    invariant 1 <= i <= powers.Length
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
  {
    powers[i] := 2 * powers[i - 1];
    assert 0 <= i - 1;
    assert powers[i - 1] == pow(2, i - 1);
    assert i != 0;
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert powers[i] == pow(2, i);

    assert forall j :: 0 <= j < i + 1 ==> powers[j] == pow(2, j) by {
      forall j | 0 <= j < i + 1
        ensures powers[j] == pow(2, j)
      {
```

```
      if j < i {
        assert powers[j] == pow(2, j);
      } else {
        assert j == i;
        assert powers[j] == pow(2, j);
      }
    }
   }
   i := i + 1;
  }
}
```

Final Program

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_array() returns (powers: array<int>)
       ensures powers.Length == 21
       ensures forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)
{
  powers := new int[21];
  powers[0] := 1;
  assert powers[0] == pow(2, 0);

  var i := 1;
  while i < powers.Length
    invariant powers.Length == 21
    invariant 1 <= i <= powers.Length
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
  {
    powers[i] := 2 * powers[i - 1];
    assert 0 <= i - 1;
    assert powers[i - 1] == pow(2, i - 1);
    assert i != 0;
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert powers[i] == pow(2, i);

    assert forall j :: 0 <= j < i + 1 ==> powers[j] == pow(2, j) by {
      forall j | 0 <= j < i + 1
        ensures powers[j] == pow(2, j)
      {
        if j < i {
          assert powers[j] == pow(2, j);
        } else {
          assert j == i;
```

```
        assert powers[j] == pow(2, j);
      }
    }
  }
  i := i + 1;
 }
}
```

Total Token Usage

Input tokens: 3051
Output tokens: 6565
Reasoning tokens: 5633
Sum of 'total tokens': 9616

Experiment Timings

Overall Experiment started at 1780651931850, ended at 1780652126501, lasting 194651ms (194.65 seconds)
Iteration #1 started at 1780651931853, ended at 1780652047894, lasting 116041ms (116.04 seconds)
Iteration #2 started at 1780652047894, ended at 1780652086616, lasting 38722ms (38.72 seconds)
Iteration #3 started at 1780652086617, ended at 1780652126501, lasting 39884ms (39.88 seconds)

Experiment 'p_4_1_c_powers_of_two_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_1_c_powers_of_two_seq
Natural language statement: null
Method signature: p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)

Ensures

- `|powers| == 21`
- `forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)`

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)

The method should respect the following contract:

ensures |powers| == 21, ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)

The contract uses the following dafny code:

function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLAERjjWvF3k97wf2ksBgsq8Ff6P
Model: gpt-5.5-2026-04-23
Created at: 1780651798
Finish reason: stop
Usage: 438 tokens in, and 1176 tokens out

```
{
  powers := [
    1,
    2,
    4,
    8,
    16,
    32,
    64,
    128,
```

```
    256,
    512,
    1024,
    2048,
    4096,
    8192,
    16384,
    32768,
    65536,
    131072,
    262144,
    524288,
    1048576
  ];
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_1_c_powers_of_two_seq/program.dfy(11,0): Error: a postcondition could
    ↪ not be proved on this return path
   |
11 | {
   | ^

problem_data/p_4_1_c_powers_of_two_seq/program.dfy(10,9): Related location: this is the
    ↪ postcondition that could not be proved
   |
10 |  ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)
   |  ^^^^^^


Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLB1aoKKGOvvbEXZNpGGZYEqOMj8
Model: gpt-5.5-2026-04-23
Created at: 1780651847
Finish reason: stop
Usage: 784 tokens in, and 3806 tokens out

```
{
  powers := [
    1,
```

```
  2,
  4,
  8,
  16,
  32,
  64,
  128,
  256,
  512,
  1024,
  2048,
  4096,
  8192,
  16384,
  32768,
  65536,
  131072,
  262144,
  524288,
  1048576
];

assert |powers| == 21;

assert pow(2, 0) == 1;
assert pow(2, 1) == 2 * pow(2, 0);
assert pow(2, 1) == 2;
assert pow(2, 2) == 2 * pow(2, 1);
assert pow(2, 2) == 4;
assert pow(2, 3) == 2 * pow(2, 2);
assert pow(2, 3) == 8;
assert pow(2, 4) == 2 * pow(2, 3);
assert pow(2, 4) == 16;
assert pow(2, 5) == 2 * pow(2, 4);
assert pow(2, 5) == 32;
assert pow(2, 6) == 2 * pow(2, 5);
assert pow(2, 6) == 64;
assert pow(2, 7) == 2 * pow(2, 6);
assert pow(2, 7) == 128;
assert pow(2, 8) == 2 * pow(2, 7);
assert pow(2, 8) == 256;
assert pow(2, 9) == 2 * pow(2, 8);
assert pow(2, 9) == 512;
assert pow(2, 10) == 2 * pow(2, 9);
assert pow(2, 10) == 1024;
assert pow(2, 11) == 2 * pow(2, 10);
assert pow(2, 11) == 2048;
assert pow(2, 12) == 2 * pow(2, 11);
assert pow(2, 12) == 4096;
assert pow(2, 13) == 2 * pow(2, 12);
assert pow(2, 13) == 8192;
```

```
assert pow(2, 14) == 2 * pow(2, 13);
assert pow(2, 14) == 16384;
assert pow(2, 15) == 2 * pow(2, 14);
assert pow(2, 15) == 32768;
assert pow(2, 16) == 2 * pow(2, 15);
assert pow(2, 16) == 65536;
assert pow(2, 17) == 2 * pow(2, 16);
assert pow(2, 17) == 131072;
assert pow(2, 18) == 2 * pow(2, 17);
assert pow(2, 18) == 262144;
assert pow(2, 19) == 2 * pow(2, 18);
assert pow(2, 19) == 524288;
assert pow(2, 20) == 2 * pow(2, 19);
assert pow(2, 20) == 1048576;

forall i | 0 <= i < |powers|
  ensures powers[i] == pow(2, i)
{
  if i == 0 {
    assert powers[i] == 1;
    assert pow(2, i) == 1;
  } else if i == 1 {
    assert powers[i] == 2;
    assert pow(2, i) == 2;
  } else if i == 2 {
    assert powers[i] == 4;
    assert pow(2, i) == 4;
  } else if i == 3 {
    assert powers[i] == 8;
    assert pow(2, i) == 8;
  } else if i == 4 {
    assert powers[i] == 16;
    assert pow(2, i) == 16;
  } else if i == 5 {
    assert powers[i] == 32;
    assert pow(2, i) == 32;
  } else if i == 6 {
    assert powers[i] == 64;
    assert pow(2, i) == 64;
  } else if i == 7 {
    assert powers[i] == 128;
    assert pow(2, i) == 128;
  } else if i == 8 {
    assert powers[i] == 256;
    assert pow(2, i) == 256;
  } else if i == 9 {
    assert powers[i] == 512;
    assert pow(2, i) == 512;
  } else if i == 10 {
    assert powers[i] == 1024;
    assert pow(2, i) == 1024;
```

```
    } else if i == 11 {
      assert powers[i] == 2048;
      assert pow(2, i) == 2048;
    } else if i == 12 {
      assert powers[i] == 4096;
      assert pow(2, i) == 4096;
    } else if i == 13 {
      assert powers[i] == 8192;
      assert pow(2, i) == 8192;
    } else if i == 14 {
      assert powers[i] == 16384;
      assert pow(2, i) == 16384;
    } else if i == 15 {
      assert powers[i] == 32768;
      assert pow(2, i) == 32768;
    } else if i == 16 {
      assert powers[i] == 65536;
      assert pow(2, i) == 65536;
    } else if i == 17 {
      assert powers[i] == 131072;
      assert pow(2, i) == 131072;
    } else if i == 18 {
      assert powers[i] == 262144;
      assert pow(2, i) == 262144;
    } else if i == 19 {
      assert powers[i] == 524288;
      assert pow(2, i) == 524288;
    } else {
      assert i == 20;
      assert powers[i] == 1048576;
      assert pow(2, i) == 1048576;
    }
  }
}
```

Final Program

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)
       ensures |powers| == 21
       ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)
{
  powers := [
    1,
```

```
  2,
  4,
  8,
  16,
  32,
  64,
  128,
  256,
  512,
  1024,
  2048,
  4096,
  8192,
  16384,
  32768,
  65536,
  131072,
  262144,
  524288,
  1048576
];

assert |powers| == 21;

assert pow(2, 0) == 1;
assert pow(2, 1) == 2 * pow(2, 0);
assert pow(2, 1) == 2;
assert pow(2, 2) == 2 * pow(2, 1);
assert pow(2, 2) == 4;
assert pow(2, 3) == 2 * pow(2, 2);
assert pow(2, 3) == 8;
assert pow(2, 4) == 2 * pow(2, 3);
assert pow(2, 4) == 16;
assert pow(2, 5) == 2 * pow(2, 4);
assert pow(2, 5) == 32;
assert pow(2, 6) == 2 * pow(2, 5);
assert pow(2, 6) == 64;
assert pow(2, 7) == 2 * pow(2, 6);
assert pow(2, 7) == 128;
assert pow(2, 8) == 2 * pow(2, 7);
assert pow(2, 8) == 256;
assert pow(2, 9) == 2 * pow(2, 8);
assert pow(2, 9) == 512;
assert pow(2, 10) == 2 * pow(2, 9);
assert pow(2, 10) == 1024;
assert pow(2, 11) == 2 * pow(2, 10);
assert pow(2, 11) == 2048;
assert pow(2, 12) == 2 * pow(2, 11);
assert pow(2, 12) == 4096;
assert pow(2, 13) == 2 * pow(2, 12);
assert pow(2, 13) == 8192;
```

```
assert pow(2, 14) == 2 * pow(2, 13);
assert pow(2, 14) == 16384;
assert pow(2, 15) == 2 * pow(2, 14);
assert pow(2, 15) == 32768;
assert pow(2, 16) == 2 * pow(2, 15);
assert pow(2, 16) == 65536;
assert pow(2, 17) == 2 * pow(2, 16);
assert pow(2, 17) == 131072;
assert pow(2, 18) == 2 * pow(2, 17);
assert pow(2, 18) == 262144;
assert pow(2, 19) == 2 * pow(2, 18);
assert pow(2, 19) == 524288;
assert pow(2, 20) == 2 * pow(2, 19);
assert pow(2, 20) == 1048576;

forall i | 0 <= i < |powers|
  ensures powers[i] == pow(2, i)
{
  if i == 0 {
    assert powers[i] == 1;
    assert pow(2, i) == 1;
  } else if i == 1 {
    assert powers[i] == 2;
    assert pow(2, i) == 2;
  } else if i == 2 {
    assert powers[i] == 4;
    assert pow(2, i) == 4;
  } else if i == 3 {
    assert powers[i] == 8;
    assert pow(2, i) == 8;
  } else if i == 4 {
    assert powers[i] == 16;
    assert pow(2, i) == 16;
  } else if i == 5 {
    assert powers[i] == 32;
    assert pow(2, i) == 32;
  } else if i == 6 {
    assert powers[i] == 64;
    assert pow(2, i) == 64;
  } else if i == 7 {
    assert powers[i] == 128;
    assert pow(2, i) == 128;
  } else if i == 8 {
    assert powers[i] == 256;
    assert pow(2, i) == 256;
  } else if i == 9 {
    assert powers[i] == 512;
    assert pow(2, i) == 512;
  } else if i == 10 {
    assert powers[i] == 1024;
    assert pow(2, i) == 1024;
```

```
    } else if i == 11 {
      assert powers[i] == 2048;
      assert pow(2, i) == 2048;
    } else if i == 12 {
      assert powers[i] == 4096;
      assert pow(2, i) == 4096;
    } else if i == 13 {
      assert powers[i] == 8192;
      assert pow(2, i) == 8192;
    } else if i == 14 {
      assert powers[i] == 16384;
      assert pow(2, i) == 16384;
    } else if i == 15 {
      assert powers[i] == 32768;
      assert pow(2, i) == 32768;
    } else if i == 16 {
      assert powers[i] == 65536;
      assert pow(2, i) == 65536;
    } else if i == 17 {
      assert powers[i] == 131072;
      assert pow(2, i) == 131072;
    } else if i == 18 {
      assert powers[i] == 262144;
      assert pow(2, i) == 262144;
    } else if i == 19 {
      assert powers[i] == 524288;
      assert pow(2, i) == 524288;
    } else {
      assert i == 20;
      assert powers[i] == 1048576;
      assert pow(2, i) == 1048576;
    }
  }
}
```

Total Token Usage

Input tokens: 1222
Output tokens: 4982
Reasoning tokens: 3169
Sum of 'total tokens': 6204

Experiment Timings

Overall Experiment started at 1780651797454, ended at 1780651930632, lasting 133178ms (133.18 seconds)
Iteration #1 started at 1780651797460, ended at 1780651846772, lasting 49312ms (49.31 seconds)
Iteration #2 started at 1780651846781, ended at 1780651930632, lasting 83851ms (83.85 seconds)

Experiment 'p_4_1_d_sum_odd_numbers' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_d_sum_odd_numbers
Natural language statement: null
Method signature: p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)

Ensures

- `sum == sum_odd(a, b)`

Functional Code Given

```
function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)

The method should respect the following contract:

ensures sum == sum_odd(a, b)

The contract uses the following dafny code:

function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLFYhoDcVepiRNr1Wg23Gx1X62av
Model: gpt-5.5-2026-04-23
Created at: 1780652128
Finish reason: stop
Usage: 444 tokens in, and 3881 tokens out

```
{
  if a > b {
    sum := 0;
    assert sum_odd(a, b) == 0;
  } else {
    sum := 0;
    var i := a;
    while i <= b
      invariant a <= i <= b + 1
```

```
    invariant sum + sum_odd(i, b) == sum_odd(a, b)
    decreases b - i + 1
  {
    if i % 2 == 1 {
      assert sum_odd(i, b) == i + sum_odd(i + 1, b);
      sum := sum + i;
    } else {
      assert sum_odd(i, b) == sum_odd(i + 1, b);
    }
    assert sum + sum_odd(i + 1, b) == sum_odd(a, b);
    i := i + 1;
  }
  assert i == b + 1;
  assert sum_odd(i, b) == 0;
  assert sum == sum_odd(a, b);
 }
}
```

Final Program

```
function sum_odd(a: int, b: int): int
decreases b - a
{
 if a > b then 0
 else if a % 2 == 1 then a + sum_odd(a + 1, b)
 else sum_odd(a + 1, b)
}

method p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)
      ensures sum == sum_odd(a, b)
{
 if a > b {
   sum := 0;
   assert sum_odd(a, b) == 0;
 } else {
   sum := 0;
   var i := a;
   while i <= b
     invariant a <= i <= b + 1
     invariant sum + sum_odd(i, b) == sum_odd(a, b)
     decreases b - i + 1
   {
     if i % 2 == 1 {
       assert sum_odd(i, b) == i + sum_odd(i + 1, b);
       sum := sum + i;
     } else {
       assert sum_odd(i, b) == sum_odd(i + 1, b);
     }
     assert sum + sum_odd(i + 1, b) == sum_odd(a, b);
     i := i + 1;
```

```
    }
    assert i == b + 1;
    assert sum_odd(i, b) == 0;
    assert sum == sum_odd(a, b);
  }
}
```

Total Token Usage

Input tokens: 444
Output tokens: 3881
Reasoning tokens: 3581
Sum of 'total tokens': 4325

Experiment Timings

Overall Experiment started at 1780652127990, ended at 1780652233621, lasting 105631ms (105.63 seconds)
Iteration #1 started at 1780652127996, ended at 1780652233621, lasting 105625ms (105.63 seconds)

Experiment 'p_4_1_e_sum_odd_digits' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_e_sum_odd_digits
Natural language statement: null
Method signature: p_4_1_e_sum_odd_digits(a: int) returns (sum: int)

Ensures

- `sum >= 0`
- `sum == s_o_d(a)`

Requires

- `a >= 0`

Functional Code Given

```
function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_1_e_sum_odd_digits(a: int) returns (sum: int)

The method should respect the following contract:

requires a >= 0, ensures sum >= 0, ensures sum == s_o_d(a)

The contract uses the following dafny code:

function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLHIxa1Heilt6LVabUUnUuj9sh1d
Model: gpt-5.5-2026-04-23
Created at: 1780652236
Finish reason: stop
Usage: 446 tokens in, and 618 tokens out

```
{
  if a == 0 {
    sum := 0;
  } else if a % 2 == 0 {
    sum := p_4_1_e_sum_odd_digits(a / 10);
  } else {
    var rest := p_4_1_e_sum_odd_digits(a / 10);
    sum := a % 10 + rest;
  }
```

```
}
```

Final Program

```
function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}

method p_4_1_e_sum_odd_digits(a: int) returns (sum: int)
       requires a >= 0
       ensures sum >= 0
       ensures sum == s_o_d(a)
{
  if a == 0 {
    sum := 0;
  } else if a % 2 == 0 {
    sum := p_4_1_e_sum_odd_digits(a / 10);
  } else {
    var rest := p_4_1_e_sum_odd_digits(a / 10);
    sum := a % 10 + rest;
  }
}
```

Total Token Usage

Input tokens: 446
Output tokens: 618
Reasoning tokens: 512
Sum of 'total tokens': 1064

Experiment Timings

Overall Experiment started at 1780652236111, ended at 1780652260060, lasting 23949ms (23.95 seconds)
Iteration #1 started at 1780652236234, ended at 1780652260060, lasting 23826ms (23.83 seconds)

Experiment 'p_4_2_a_min_max_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_a_min_max_array
Natural language statement: null
Method signature: p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)

Ensures

- `forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]`
- `forall i :: 0 <= i < inputs.Length ==> max >= inputs[i]`
- `exists i :: 0 <= i < inputs.Length && min == inputs[i]`
- `exists i :: 0 <= i < inputs.Length && max == inputs[i]`

Requires

- `0 < inputs.Length`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)

The method should respect the following contract:

requires 0 < inputs.Length, ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[
    ↪ i], ensures forall i :: 0 <= i < inputs.Length ==> max >= inputs[i], ensures
    ↪ exists i :: 0 <= i < inputs.Length && min == inputs[i], ensures exists i :: 0 <= i
    ↪  < inputs.Length && max == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLI7toHsksmHlKuFbylkLNi2MKWj
Model: gpt-5.5-2026-04-23
Created at: 1780652287
Finish reason: stop
Usage: 392 tokens in, and 736 tokens out

```
{
  min := inputs[0];
  max := inputs[0];
  var idxMin := 0;
  var idxMax := 0;
  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant 0 <= idxMin < i
    invariant 0 <= idxMax < i
    invariant min == inputs[idxMin]
    invariant max == inputs[idxMax]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
      idxMin := i;
```

```
    }
    if inputs[i] > max {
      max := inputs[i];
      idxMax := i;
    }
    i := i + 1;
  }
}
```

Final Program

```
method p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)
       requires 0 < inputs.Length
       ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]
       ensures forall i :: 0 <= i < inputs.Length ==> max >= inputs[i]
       ensures exists i :: 0 <= i < inputs.Length && min == inputs[i]
       ensures exists i :: 0 <= i < inputs.Length && max == inputs[i]
{
  min := inputs[0];
  max := inputs[0];
  var idxMin := 0;
  var idxMax := 0;
  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant 0 <= idxMin < i
    invariant 0 <= idxMax < i
    invariant min == inputs[idxMin]
    invariant max == inputs[idxMax]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
      idxMin := i;
    }
    if inputs[i] > max {
      max := inputs[i];
      idxMax := i;
    }
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 392
Output tokens: 736

Reasoning tokens: 512
Sum of 'total tokens': 1128

Experiment Timings

Overall Experiment started at 1780652287040, ended at 1780652312103, lasting 25063ms (25.06 seconds)
Iteration #1 started at 1780652287041, ended at 1780652312103, lasting 25062ms (25.06 seconds)

Experiment 'p_4_2_a_min_max_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_a_min_max_seq
Natural language statement: null
Method signature: p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)

Ensures

- `forall i :: 0 <= i < |inputs| ==> min <= inputs[i]`
- `forall i :: 0 <= i < |inputs| ==> max >= inputs[i]`
- `exists i :: 0 <= i < |inputs| && min == inputs[i]`
- `exists i :: 0 <= i < |inputs| && max == inputs[i]`

Requires

- `0 < |inputs|`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)

The method should respect the following contract:

requires 0 < |inputs|, ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i],
    ↪ ensures forall i :: 0 <= i < |inputs| ==> max >= inputs[i], ensures exists i :: 0
    ↪ <= i < |inputs| && min == inputs[i], ensures exists i :: 0 <= i < |inputs| && max
    ↪ == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLHi7ycIFUEY4As2PRZ5MuRpNU4z
Model: gpt-5.5-2026-04-23
Created at: 1780652262
Finish reason: stop
Usage: 402 tokens in, and 751 tokens out

```
{
  min := inputs[0];
  max := inputs[0];
  ghost var minIdx := 0;
  ghost var maxIdx := 0;
  var j := 1;

  while j < |inputs|
    invariant 1 <= j <= |inputs|
    invariant 0 <= minIdx < j
    invariant 0 <= maxIdx < j
    invariant min == inputs[minIdx]
    invariant max == inputs[maxIdx]
    invariant forall i :: 0 <= i < j ==> min <= inputs[i]
    invariant forall i :: 0 <= i < j ==> max >= inputs[i]
    decreases |inputs| - j
  {
    if inputs[j] < min {
```

```
    min := inputs[j];
    minIdx := j;
  }

  if inputs[j] > max {
    max := inputs[j];
    maxIdx := j;
  }

  j := j + 1;
 }
}
```

Final Program

```
method p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)
      requires 0 < |inputs|
      ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
      ensures forall i :: 0 <= i < |inputs| ==> max >= inputs[i]
      ensures exists i :: 0 <= i < |inputs| && min == inputs[i]
      ensures exists i :: 0 <= i < |inputs| && max == inputs[i]
{
 min := inputs[0];
 max := inputs[0];
 ghost var minIdx := 0;
 ghost var maxIdx := 0;
 var j := 1;

 while j < |inputs|
   invariant 1 <= j <= |inputs|
   invariant 0 <= minIdx < j
   invariant 0 <= maxIdx < j
   invariant min == inputs[minIdx]
   invariant max == inputs[maxIdx]
   invariant forall i :: 0 <= i < j ==> min <= inputs[i]
   invariant forall i :: 0 <= i < j ==> max >= inputs[i]
   decreases |inputs| - j
 {
   if inputs[j] < min {
     min := inputs[j];
     minIdx := j;
   }

   if inputs[j] > max {
     max := inputs[j];
     maxIdx := j;
   }

   j := j + 1;
 }
```

```
}
```

Total Token Usage

Input tokens: 402
Output tokens: 751
Reasoning tokens: 512
Sum of 'total tokens': 1153

Experiment Timings

Overall Experiment started at 1780652261776, ended at 1780652286200, lasting 24424ms (24.42 seconds)
Iteration #1 started at 1780652261794, ended at 1780652286200, lasting 24406ms (24.41 seconds)

Experiment 'p_4_2_b_count_even_odd_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_b_count_even_odd_array
Natural language statement: null
Method signature: p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int, odd_count: int)

Ensures

- `even_count == number_even(inputs[..])`
- `odd_count == number_odd(inputs[..])`

Functional Code Given

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int, odd_count: int
    ↪ )

The method should respect the following contract:

ensures even_count == number_even(inputs[..]), ensures odd_count == number_odd(inputs
    ↪ [..])

The contract uses the following dafny code:

function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLJIg01p1kT023uaC0vXOJCv6Wiq
Model: gpt-5.5-2026-04-23
Created at: 1780652360
Finish reason: stop
Usage: 558 tokens in, and 3619 tokens out

```
{
  even_count := 0;
  odd_count := 0;
```

```
  var i := inputs.Length;

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant even_count == number_even(inputs[i..])
    invariant odd_count == number_odd(inputs[i..])
    decreases i
  {
    i := i - 1;
    assert 0 <= i < inputs.Length;
    assert even_count == number_even(inputs[i + 1..]);
    assert odd_count == number_odd(inputs[i + 1..]);
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert number_even(inputs[i..]) == 1 + number_even(inputs[i + 1..]);
      assert number_odd(inputs[i..]) == number_odd(inputs[i + 1..]);
      even_count := even_count + 1;
    } else {
      assert 0 <= inputs[i] % 2 < 2;
      assert inputs[i] % 2 == 1;
      assert number_even(inputs[i..]) == number_even(inputs[i + 1..]);
      assert number_odd(inputs[i..]) == 1 + number_odd(inputs[i + 1..]);
      odd_count := odd_count + 1;
    }
  }

  assert i == 0;
  assert inputs[i..] == inputs[..];
}
```

Final Program

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

method p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int,
    ↪ odd_count: int)
       ensures even_count == number_even(inputs[..])
       ensures odd_count == number_odd(inputs[..])
```

```
{
  even_count := 0;
  odd_count := 0;
  var i := inputs.Length;

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant even_count == number_even(inputs[i..])
    invariant odd_count == number_odd(inputs[i..])
    decreases i
  {
    i := i - 1;
    assert 0 <= i < inputs.Length;
    assert even_count == number_even(inputs[i + 1..]);
    assert odd_count == number_odd(inputs[i + 1..]);
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert number_even(inputs[i..]) == 1 + number_even(inputs[i + 1..]);
      assert number_odd(inputs[i..]) == number_odd(inputs[i + 1..]);
      even_count := even_count + 1;
    } else {
      assert 0 <= inputs[i] % 2 < 2;
      assert inputs[i] % 2 == 1;
      assert number_even(inputs[i..]) == number_even(inputs[i + 1..]);
      assert number_odd(inputs[i..]) == 1 + number_odd(inputs[i + 1..]);
      odd_count := odd_count + 1;
    }
  }

  assert i == 0;
  assert inputs[i..] == inputs[..];
}
```

Total Token Usage

Input tokens: 558
Output tokens: 3619
Reasoning tokens: 3214
Sum of 'total tokens': 4177

Experiment Timings

Overall Experiment started at 1780652360235, ended at 1780652452562, lasting 92327ms (92.33 seconds)
Iteration #1 started at 1780652360240, ended at 1780652452562, lasting 92322ms (92.32 seconds)

Experiment 'p_4_2_b_count_even_odd_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_b_count_even_odd_seq
Natural language statement: null
Method signature: p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

Ensures

- `even_count == number_even(inputs)`
- `odd_count == number_odd(inputs)`

Functional Code Given

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

The method should respect the following contract:

ensures even_count == number_even(inputs), ensures odd_count == number_odd(inputs)

The contract uses the following dafny code:

function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLIXJSgSQnOJueilQg4XOyxHZLhp
Model: gpt-5.5-2026-04-23
Created at: 1780652313
Finish reason: stop
Usage: 554 tokens in, and 1184 tokens out

```
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
  } else {
```

```
    var tail_even, tail_odd := p_4_2_b_count_even_odd_seq(inputs[1..]);
    if inputs[0] % 2 == 0 {
      even_count := tail_even + 1;
      odd_count := tail_odd;
    } else {
      assert inputs[0] % 2 == 1;
      even_count := tail_even;
      odd_count := tail_odd + 1;
    }
  }
}
```

Final Program

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

method p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count:
    ↪ int)
       ensures even_count == number_even(inputs)
       ensures odd_count == number_odd(inputs)
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
  } else {
    var tail_even, tail_odd := p_4_2_b_count_even_odd_seq(inputs[1..]);
    if inputs[0] % 2 == 0 {
      even_count := tail_even + 1;
      odd_count := tail_odd;
    } else {
      assert inputs[0] % 2 == 1;
      even_count := tail_even;
      odd_count := tail_odd + 1;
    }
  }
}
```

Total Token Usage

Input tokens: 554
Output tokens: 1184
Reasoning tokens: 1024
Sum of 'total tokens': 1738

Experiment Timings

Overall Experiment started at 1780652313150, ended at 1780652358602, lasting 45452ms (45.45 seconds)
Iteration #1 started at 1780652313185, ended at 1780652358601, lasting 45416ms (45.42 seconds)

Experiment 'p_4_2_c_cumulative_totals_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 3

Problem Specification

Problem name: p_4_2_c_cumulative_totals_array
Natural language statement: null
Method signature: p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)

Ensures

- `totals.Length == inputs.Length`
- `forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])`

Requires

- `0 < inputs.Length`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)

The method should respect the following contract:

requires 0 < inputs.Length, ensures totals.Length == inputs.Length, ensures forall i :: 0
    ↪  <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs[0] else totals[i -
    ↪ 1] + inputs[i])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLNBDqeH8HJyGG1tMvD9tay45fRM
Model: gpt-5.5-2026-04-23
Created at: 1780652601
Finish reason: stop
Usage: 348 tokens in, and 640 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
        ↪ totals[j - 1] + inputs[j])
  {
    totals[i] := totals[i - 1] + inputs[i];
    i := i + 1;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_2_c_cumulative_totals_array/program.dfy(13,14): Error: this invariant
    ↪ could not be proved to be maintained by the loop
 Related message: loop invariant violation
   |
13 | invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
    ↪ totals[j - 1] + inputs[j])
   | ^^^^^^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLNbYzcdtpTyKgjCl27MsGjYUraQ
Model: gpt-5.5-2026-04-23
Created at: 1780652627
Finish reason: stop
Usage: 694 tokens in, and 3635 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
        ↪ totals[j - 1] + inputs[j])
  {
    var oldI := i;
    ghost var prev := totals[..oldI];

    assert forall j :: 0 <= j < oldI ==> prev[j] == (if j == 0 then inputs[0] else prev[j
        ↪ - 1] + inputs[j]) by {
      forall j | 0 <= j < oldI
        ensures prev[j] == (if j == 0 then inputs[0] else prev[j - 1] + inputs[j])
      {
        assert prev[j] == totals[j];
        if j == 0 {
          assert totals[j] == inputs[0];
        } else {
          assert 0 <= j - 1 < oldI;
          assert prev[j - 1] == totals[j - 1];
```

```
        assert totals[j] == totals[j - 1] + inputs[j];
      }
    }
  }

  totals[oldI] := totals[oldI - 1] + inputs[oldI];
  assert totals[..oldI] == prev;

  i := oldI + 1;

  assert forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else totals[j
      ↪  - 1] + inputs[j]) by {
    forall j | 0 <= j < i
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      if j == oldI {
        assert j != 0;
        assert totals[j] == totals[j - 1] + inputs[j];
      } else {
        assert j < oldI;
        assert totals[j] == prev[j];
        if j == 0 {
          assert prev[j] == inputs[0];
        } else {
          assert 0 <= j - 1 < oldI;
          assert totals[j - 1] == prev[j - 1];
          assert prev[j] == prev[j - 1] + inputs[j];
        }
      }
    }
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_2_c_cumulative_totals_array/program.dfy(24,10): Error: assertion might
    ↪ not hold
   |
24 | assert totals[j] == inputs[0];
   | ^^^^^^

problem_data/p_4_2_c_cumulative_totals_array/program.dfy(49,12): Error: assertion might
    ↪ not hold
   |
49 | assert prev[j] == inputs[0];
   | ^^^^^^
```

```
Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLOpEtpB931e6Of4XDi18B7N0K8Y
Model: gpt-5.5-2026-04-23
Created at: 1780652703
Finish reason: stop
Usage: 1482 tokens in, and 2543 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    var n := i;
    totals[n] := totals[n - 1] + inputs[n];

    assert n != 0;
    assert totals[0] == inputs[0];

    assert forall j :: 1 <= j < n + 1 ==> totals[j] == totals[j - 1] + inputs[j] by {
      forall j | 1 <= j < n + 1
        ensures totals[j] == totals[j - 1] + inputs[j]
      {
        if j == n {
          assert totals[j] == totals[j - 1] + inputs[j];
        } else {
          assert j < n;
          assert 1 <= j < i;
          assert totals[j] == totals[j - 1] + inputs[j];
        }
      }
    }

    i := n + 1;
  }

  assert i == inputs.Length;
  assert forall k :: 0 <= k < inputs.Length ==> totals[k] == (if k == 0 then inputs[0]
      ↪ else totals[k - 1] + inputs[k]) by {
```

```
    forall k | 0 <= k < inputs.Length
      ensures totals[k] == (if k == 0 then inputs[0] else totals[k - 1] + inputs[k])
    {
      if k == 0 {
        assert totals[k] == inputs[0];
      } else {
        assert 1 <= k < i;
        assert totals[k] == totals[k - 1] + inputs[k];
      }
    }
  }
}
```

Final Program

```
method p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)
       requires 0 < inputs.Length
       ensures totals.Length == inputs.Length
       ensures forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs
           ↪ [0] else totals[i - 1] + inputs[i])
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    var n := i;
    totals[n] := totals[n - 1] + inputs[n];

    assert n != 0;
    assert totals[0] == inputs[0];

    assert forall j :: 1 <= j < n + 1 ==> totals[j] == totals[j - 1] + inputs[j] by {
      forall j | 1 <= j < n + 1
        ensures totals[j] == totals[j - 1] + inputs[j]
      {
        if j == n {
          assert totals[j] == totals[j - 1] + inputs[j];
        } else {
          assert j < n;
          assert 1 <= j < i;
          assert totals[j] == totals[j - 1] + inputs[j];
        }
      }
    }
```

```
    i := n + 1;
  }

  assert i == inputs.Length;
  assert forall k :: 0 <= k < inputs.Length ==> totals[k] == (if k == 0 then inputs[0]
      ↪ else totals[k - 1] + inputs[k]) by {
    forall k | 0 <= k < inputs.Length
      ensures totals[k] == (if k == 0 then inputs[0] else totals[k - 1] + inputs[k])
    {
      if k == 0 {
        assert totals[k] == inputs[0];
      } else {
        assert 1 <= k < i;
        assert totals[k] == totals[k - 1] + inputs[k];
      }
    }
  }
}
```

Total Token Usage

Input tokens: 2524
Output tokens: 6818
Reasoning tokens: 5544
Sum of 'total tokens': 9342

Experiment Timings

Overall Experiment started at 1780652600974, ended at 1780652769159, lasting 168185ms (168.19 seconds)
Iteration #1 started at 1780652600975, ended at 1780652626693, lasting 25718ms (25.72 seconds)
Iteration #2 started at 1780652626693, ended at 1780652703361, lasting 76668ms (76.67 seconds)
Iteration #3 started at 1780652703362, ended at 1780652769159, lasting 65797ms (65.80 seconds)

Experiment 'p_4_2_c_cumulative_totals_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_2_c_cumulative_totals_seq
Natural language statement: null
Method signature: p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)

Ensures

- `|totals| == |inputs|`
- `forall i :: 0 <= i < |inputs| ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])`

Requires

- `0 < |inputs|`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)

The method should respect the following contract:

requires 0 < |inputs|, ensures |totals| == |inputs|, ensures forall i :: 0 <= i < |inputs
    ↪ | ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLKpSn6At7YODDa4zUcJ5Ytli0VN
Model: gpt-5.5-2026-04-23
Created at: 1780652455
Finish reason: stop
Usage: 358 tokens in, and 3124 tokens out

```
{
  totals := [inputs[0]];
  var n := |inputs|;
  var k := 1;

  while k < n
    invariant n == |inputs|
    invariant 1 <= k <= n
    invariant |totals| == k
    invariant forall j :: 0 <= j < k ==> totals[j] == (if j == 0 then inputs[0] else
        ↪ totals[j - 1] + inputs[j])
    decreases n - k
  {
    var prevTotals := totals;
    assert |prevTotals| == k;
    assert forall j :: 0 <= j < k ==> prevTotals[j] == (if j == 0 then inputs[0] else
        ↪ prevTotals[j - 1] + inputs[j]);

    var next := prevTotals[k - 1] + inputs[k];
    totals := prevTotals + [next];
```

```
    forall j | 0 <= j < k
      ensures totals[j] == prevTotals[j]
    {
      assert 0 <= j < |prevTotals|;
      assert totals == prevTotals + [next];
      assert totals[j] == (prevTotals + [next])[j];
      assert (prevTotals + [next])[j] == prevTotals[j];
    }

    forall j | 0 <= j < k + 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      if j < k {
        assert totals[j] == prevTotals[j];
        if j == 0 {
          assert prevTotals[j] == inputs[0];
        } else {
          assert 0 <= j - 1 < k;
          assert totals[j - 1] == prevTotals[j - 1];
          assert prevTotals[j] == prevTotals[j - 1] + inputs[j];
        }
      } else {
        assert j == k;
        assert 0 < j;
        assert j - 1 == k - 1;
        assert totals[j] == next;
        assert totals[j - 1] == prevTotals[k - 1];
        assert next == prevTotals[k - 1] + inputs[k];
      }
    }

    k := k + 1;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_2_c_cumulative_totals_seq/program.dfy(39,10): Error: assertion might not
    ↪ hold
   |
39 | assert prevTotals[j] == inputs[0];
   | ^^^^^^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
```

```
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLM0tlHLFS8avdfY8Vn8bH6L6LHg
Model: gpt-5.5-2026-04-23
Created at: 1780652528
Finish reason: stop
Usage: 1016 tokens in, and 2873 tokens out

```
{
  totals := [inputs[0]];
  var n := |inputs|;
  var k := 1;

  while k < n
    invariant n == |inputs|
    invariant 1 <= k <= n
    invariant |totals| == k
    invariant forall j :: 0 <= j < k ==> totals[j] == (if j == 0 then inputs[0] else
        ↪ totals[j - 1] + inputs[j])
    decreases n - k
  {
    var prevTotals := totals;
    assert |prevTotals| == k;
    assert forall j :: 0 <= j < k ==> prevTotals[j] == (if j == 0 then inputs[0] else
        ↪ prevTotals[j - 1] + inputs[j]);

    var next := prevTotals[k - 1] + inputs[k];
    totals := prevTotals + [next];

    forall j | 0 <= j < k
      ensures totals[j] == prevTotals[j]
    {
      assert 0 <= j < |prevTotals|;
      assert totals == prevTotals + [next];
      assert totals[j] == (prevTotals + [next])[j];
      assert (prevTotals + [next])[j] == prevTotals[j];
    }

    forall j | 0 <= j < k + 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      if j < k {
        assert 0 <= j < k;
        assert totals[j] == prevTotals[j];
        assert prevTotals[j] == (if j == 0 then inputs[0] else prevTotals[j - 1] + inputs[
            ↪ j]);
        if j == 0 {
          assert prevTotals[j] == inputs[0];
```

```
      } else {
        assert 0 <= j - 1 < k;
        assert totals[j - 1] == prevTotals[j - 1];
        assert prevTotals[j] == prevTotals[j - 1] + inputs[j];
      }
    } else {
      assert j == k;
      assert 0 < j;
      assert j - 1 == k - 1;
      assert totals[j] == next;
      assert totals[j - 1] == prevTotals[k - 1];
      assert next == prevTotals[k - 1] + inputs[k];
    }
  }

  k := k + 1;
 }
}
```

Final Program

```
method p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)
      requires 0 < |inputs|
      ensures |totals| == |inputs|
      ensures forall i :: 0 <= i < |inputs| ==> totals[i] == (if i == 0 then inputs[0]
          ↪ else totals[i - 1] + inputs[i])
{
 totals := [inputs[0]];
 var n := |inputs|;
 var k := 1;

 while k < n
   invariant n == |inputs|
   invariant 1 <= k <= n
   invariant |totals| == k
   invariant forall j :: 0 <= j < k ==> totals[j] == (if j == 0 then inputs[0] else
       ↪ totals[j - 1] + inputs[j])
   decreases n - k
 {
   var prevTotals := totals;
   assert |prevTotals| == k;
   assert forall j :: 0 <= j < k ==> prevTotals[j] == (if j == 0 then inputs[0] else
       ↪ prevTotals[j - 1] + inputs[j]);

   var next := prevTotals[k - 1] + inputs[k];
   totals := prevTotals + [next];

   forall j | 0 <= j < k
     ensures totals[j] == prevTotals[j]
   {
```

```
    assert 0 <= j < |prevTotals|;
    assert totals == prevTotals + [next];
    assert totals[j] == (prevTotals + [next])[j];
    assert (prevTotals + [next])[j] == prevTotals[j];
  }

  forall j | 0 <= j < k + 1
    ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
  {
    if j < k {
      assert 0 <= j < k;
      assert totals[j] == prevTotals[j];
      assert prevTotals[j] == (if j == 0 then inputs[0] else prevTotals[j - 1] + inputs[
          ↪ j]);
      if j == 0 {
        assert prevTotals[j] == inputs[0];
      } else {
        assert 0 <= j - 1 < k;
        assert totals[j - 1] == prevTotals[j - 1];
        assert prevTotals[j] == prevTotals[j - 1] + inputs[j];
      }
    } else {
      assert j == k;
      assert 0 < j;
      assert j - 1 == k - 1;
      assert totals[j] == next;
      assert totals[j - 1] == prevTotals[k - 1];
      assert next == prevTotals[k - 1] + inputs[k];
    }
  }

  k := k + 1;
 }
}
```

Total Token Usage

Input tokens: 1374
Output tokens: 5997
Reasoning tokens: 4807
Sum of 'total tokens': 7371

Experiment Timings

Overall Experiment started at 1780652454921, ended at 1780652599541, lasting 144620ms (144.62 seconds)
Iteration #1 started at 1780652454921, ended at 1780652528040, lasting 73119ms (73.12 seconds)
Iteration #2 started at 1780652528040, ended at 1780652599540, lasting 71500ms (71.50 seconds)

Experiment 'p_4_2_d_adjacent_duplicates_array_without_4th_ensures' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 4
Hard fails (resolution): 1
Soft fails (resolution): 1
Verification attempts: 2

Problem Specification

Problem name: p_4_2_d_adjacent_duplicates_array_without_4th_ensures
Natural language statement: null
Method signature: p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (duplicates: array<int>)

Ensures

- `forall d :: d in duplicates[..] ==> d in inputs[..]`
- `forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..]`
- `forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] && inputs[i] == d`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (
    ↪ duplicates: array<int>)

The method should respect the following contract:

ensures forall d :: d in duplicates[..] ==> d in inputs[..], ensures forall i :: 0 <= i <
    ↪  inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..],
    ↪ ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
    ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLh4ZElLp4o77897TuNrSDZSchJO
Model: gpt-5.5-2026-04-23
Created at: 1780653834
Finish reason: stop
Usage: 434 tokens in, and 4788 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j < inputs.Length - 1 &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - 1 - i
  {
    var nextI := i + 1;

    if inputs[i] == inputs[i + 1] {
      var oldAcc := acc;
      acc := oldAcc + [inputs[i]];
```

```
  assert inputs[i] in acc;

  assert forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    if d in oldAcc {
      assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j
          ↪ + 1] && inputs[j] == d;
      assert forall j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j
          ↪ + 1] && inputs[j] == d
        ==> 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j + 1] &&
            ↪ inputs[j] == d;
    } else {
      assert d in [inputs[i]];
      assert d == inputs[i];
      assert 0 <= i < nextI;
      assert i < inputs.Length - 1;
      assert exists j :: 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] ==
          ↪ inputs[j + 1] && inputs[j] == d;
    }
  }

  assert forall j | 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j +
      ↪  1]
    ensures inputs[j] in acc
  {
    if j < i {
      assert inputs[j] in oldAcc;
      assert inputs[j] in acc;
    } else {
      assert j == i;
      assert inputs[j] == inputs[i];
      assert inputs[j] in acc;
    }
  }
} else {
  assert forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j +
        ↪ 1] && inputs[j] == d;
    assert forall j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j +
        ↪ 1] && inputs[j] == d
      ==> 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d;
  }

  assert forall j | 0 <= j < nextI && j < inputs.Length - 1 && inputs[j] == inputs[j +
```

```
        ↪ 1]
      ensures inputs[j] in acc
    {
      if j < i {
        assert inputs[j] in acc;
      } else {
        assert j == i;
        assert inputs[i] != inputs[i + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
 }

 duplicates := new int[|acc|](j => acc[j]);
 assert duplicates.Length == |acc|;
 assert forall j :: 0 <= j < duplicates.Length ==> duplicates[j] == acc[j];
 assert duplicates[..] == acc;

 assert forall j :: 0 <= j < inputs.Length - 1 ==> j < i;

 assert forall d | d in duplicates[..]
   ensures d in inputs[..]
 {
   assert d in acc;
   assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
       ↪ && inputs[j] == d;
 }

 assert forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
   ensures inputs[j] in duplicates[..]
 {
   assert j < i;
   assert inputs[j] in acc;
 }

 assert forall d | d in duplicates[..]
   ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
       ↪ [j] == d
 {
   assert d in acc;
   assert exists j :: 0 <= j < i && j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
       ↪ && inputs[j] == d;
 }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(51,4):
    ↪ Error: invalid UnaryExpression
   |
51 | } else {
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(50,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
50 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(50,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
50 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(72,4):
    ↪ Error: invalid UnaryExpression
   |
72 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(71,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
71 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(71,6):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
   |
71 | }
   | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(104,0):
    ↪ Error: invalid UnaryExpression
    |
104 | }
    | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(103,2):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
    |
103 | }
    | ^
```

```
problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(103,2):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
    |
103 | }
    | ^

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(103,2):
    ↪ Error: expected either 'by' or a semicolon following the assert expression
    |
103 | }
    | ^

10 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLie4OE3lIevzRNRzhVZRgvqiyNW
Model: gpt-5.5-2026-04-23
Created at: 1780653932
Finish reason: stop
Usage: 2593 tokens in, and 6074 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - i
  {
    var oldI := i;
    var nextI := i + 1;

    if inputs[oldI] == inputs[oldI + 1] {
      var oldAcc := acc;
      assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
          ↪  1] ==> inputs[j] in oldAcc;
      assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
          ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

      acc := oldAcc + [inputs[oldI]];
```

```
  forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    assert d in oldAcc + [inputs[oldI]];
    assert d in oldAcc || d in [inputs[oldI]];
    if d in oldAcc {
      var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
          ↪  inputs[j] == d;
      assert 0 <= j < nextI;
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
          ↪ inputs[k + 1] && inputs[k] == d;
    } else {
      assert d in [inputs[oldI]];
      assert d == inputs[oldI];
      assert 0 <= oldI < nextI;
      assert oldI + 1 < inputs.Length;
      assert inputs[oldI] == inputs[oldI + 1];
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
          ↪ inputs[k + 1] && inputs[k] == d;
    }
  }

  forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
    ensures inputs[j] in acc
  {
    if j < oldI {
      assert inputs[j] in oldAcc;
      assert inputs[j] in acc;
    } else {
      assert j == oldI;
      assert inputs[j] == inputs[oldI];
      assert inputs[oldI] in [inputs[oldI]];
      assert inputs[oldI] in acc;
      assert inputs[j] in acc;
    }
  }
} else {
  forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
        ↪ inputs[j] == d;
    assert 0 <= j < nextI;
    assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
        ↪ k + 1] && inputs[k] == d;
  }

  forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
    ensures inputs[j] in acc
```

```
    {
      if j < oldI {
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[oldI] != inputs[oldI + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
}

duplicates := new int[|acc|](j => acc[j]);
assert duplicates.Length == |acc|;
assert |duplicates[..]| == |acc|;
forall j | 0 <= j < |acc|
  ensures duplicates[..][j] == acc[j]
{
  assert duplicates[..][j] == duplicates[j];
  assert duplicates[j] == acc[j];
}
assert duplicates[..] == acc;

forall j | 0 <= j < inputs.Length - 1
  ensures j < i
{
  assert !(i + 1 < inputs.Length);
  assert i + 1 >= inputs.Length;
  assert j + 1 < inputs.Length;
  assert j < i;
}

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in acc;
  var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
      ↪ ] == d;
  assert 0 <= j < inputs.Length;
  assert inputs[j] == d;
  assert inputs[j] in inputs[..];
}

forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates[..]
{
  assert j < i;
  assert j + 1 < inputs.Length;
```

```
    assert inputs[j] in acc;
    assert inputs[j] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
        ↪ [j] == d
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert j < inputs.Length - 1;
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(95,2):
    ↪ Warning: Could not find a trigger for this quantifier. Without a trigger, the
    ↪ quantifier may cause brittle verification. To silence this warning, add an
    ↪ explicit trigger using the {:trigger} attribute. For more information, see the
    ↪ section on quantifier instantiation rules in the reference manual.
   |
95 | forall j | 0 <= j < inputs.Length - 1
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLkX4x29QM7hK3MeRG9bHtLpEgYB
Model: gpt-5.5-2026-04-23
Created at: 1780654049
Finish reason: stop
Usage: 3902 tokens in, and 2336 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
```

```
  invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
      ↪ 1] ==> inputs[j] in acc
  invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
      ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
  decreases inputs.Length - i
{
  var oldI := i;
  var nextI := i + 1;

  if inputs[oldI] == inputs[oldI + 1] {
    var oldAcc := acc;
    assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪  1] ==> inputs[j] in oldAcc;
    assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
        ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

    acc := oldAcc + [inputs[oldI]];

    forall d | d in acc
      ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
          ↪ [j + 1] && inputs[j] == d
    {
      assert d in oldAcc + [inputs[oldI]];
      assert d in oldAcc || d in [inputs[oldI]];
      if d in oldAcc {
        var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
            ↪  inputs[j] == d;
        assert 0 <= j < nextI;
        assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
            ↪ inputs[k + 1] && inputs[k] == d;
      } else {
        assert d in [inputs[oldI]];
        assert d == inputs[oldI];
        assert 0 <= oldI < nextI;
        assert oldI + 1 < inputs.Length;
        assert inputs[oldI] == inputs[oldI + 1];
        assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
            ↪ inputs[k + 1] && inputs[k] == d;
      }
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in oldAcc;
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[j] == inputs[oldI];
        assert inputs[oldI] in [inputs[oldI]];
```

```
      assert inputs[oldI] in acc;
      assert inputs[j] in acc;
    }
  }
} else {
  forall d | d in acc
    ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
        ↪ [j + 1] && inputs[j] == d
  {
    var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
        ↪ inputs[j] == d;
    assert 0 <= j < nextI;
    assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
        ↪ k + 1] && inputs[k] == d;
  }

  forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
    ensures inputs[j] in acc
  {
    if j < oldI {
      assert inputs[j] in acc;
    } else {
      assert j == oldI;
      assert inputs[oldI] != inputs[oldI + 1];
      assert inputs[j] == inputs[j + 1];
      assert false;
    }
  }
}

i := nextI;
}

duplicates := new int[|acc|](j => acc[j]);
assert duplicates.Length == |acc|;
assert |duplicates[..]| == |acc|;
forall j | 0 <= j < |acc|
  ensures duplicates[..][j] == acc[j]
{
  assert duplicates[..][j] == duplicates[j];
  assert duplicates[j] == acc[j];
}
assert duplicates[..] == acc;

assert !(i + 1 < inputs.Length);
assert i + 1 >= inputs.Length;

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in acc;
```

```
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert 0 <= j < inputs.Length;
    assert inputs[j] == d;
    assert inputs[j] in inputs[..];
  }

  forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates[..]
  {
    assert j + 1 < inputs.Length;
    assert j + 1 < i + 1;
    assert j < i;
    assert inputs[j] in acc;
    assert inputs[j] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
        ↪ [j] == d
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert j < inputs.Length - 1;
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(84,39):
    ↪ Error: index out of range
   |
84 | duplicates := new int[|acc|](j => acc[j]);
   |                                       ^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLlWG9hoUWpQV1DTNUYGOqwXzVxu

Model: gpt-5.5-2026-04-23
Created at: 1780654110
Finish reason: stop
Usage: 5460 tokens in, and 2541 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - i
  {
    var oldI := i;
    var nextI := i + 1;

    if inputs[oldI] == inputs[oldI + 1] {
      var oldAcc := acc;
      assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
          ↪  1] ==> inputs[j] in oldAcc;
      assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
          ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

      acc := oldAcc + [inputs[oldI]];

      forall d | d in acc
        ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
            ↪ [j + 1] && inputs[j] == d
      {
        assert d in oldAcc + [inputs[oldI]];
        assert d in oldAcc || d in [inputs[oldI]];
        if d in oldAcc {
          var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
              ↪  inputs[j] == d;
          assert 0 <= j < nextI;
          assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
              ↪ inputs[k + 1] && inputs[k] == d;
        } else {
          assert d in [inputs[oldI]];
          assert d == inputs[oldI];
          assert 0 <= oldI < nextI;
          assert oldI + 1 < inputs.Length;
          assert inputs[oldI] == inputs[oldI + 1];
          assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
              ↪ inputs[k + 1] && inputs[k] == d;
        }
      }
```

```
    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in oldAcc;
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[j] == inputs[oldI];
        assert inputs[oldI] in [inputs[oldI]];
        assert inputs[oldI] in acc;
        assert inputs[j] in acc;
      }
    }
  } else {
    forall d | d in acc
      ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
          ↪ [j + 1] && inputs[j] == d
    {
      var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d;
      assert 0 <= j < nextI;
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
          ↪ k + 1] && inputs[k] == d;
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[oldI] != inputs[oldI + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
}

duplicates := new int[|acc|];
var k := 0;
while k < |acc|
  invariant 0 <= k <= |acc|
  invariant duplicates.Length == |acc|
  invariant forall t :: 0 <= t < k ==> duplicates[t] == acc[t]
  decreases |acc| - k
{
```

```
    duplicates[k] := acc[k];
    k := k + 1;
  }

  assert duplicates.Length == |acc|;
  assert |duplicates[..]| == |acc|;
  forall t | 0 <= t < |acc|
    ensures duplicates[..][t] == acc[t]
  {
    assert duplicates[..][t] == duplicates[t];
    assert duplicates[t] == acc[t];
  }
  assert duplicates[..] == acc;

  assert !(i + 1 < inputs.Length);
  assert i + 1 >= inputs.Length;

  forall d | d in duplicates[..]
    ensures d in inputs[..]
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert 0 <= j < inputs.Length;
    assert inputs[j] == d;
    assert inputs[j] in inputs[..];
  }

  forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates[..]
  {
    assert j + 1 < inputs.Length;
    assert j + 1 < i + 1;
    assert j < i;
    assert inputs[j] in acc;
    assert inputs[j] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
        ↪ [j] == d
  {
    assert d in acc;
    var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
        ↪ ] == d;
    assert j < inputs.Length - 1;
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Final Program

```
method p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns
    ↪ (duplicates: array<int>)
      ensures forall d :: d in duplicates[..] ==> d in inputs[..]
      ensures forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==>
          ↪ inputs[i] in duplicates[..]
      ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
          ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d
{
  var acc: seq<int> := [];
  var i := 0;

  while i + 1 < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j +
        ↪ 1] ==> inputs[j] in acc
    invariant forall d :: d in acc ==> exists j :: 0 <= j < i && j + 1 < inputs.Length &&
        ↪ inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases inputs.Length - i
  {
    var oldI := i;
    var nextI := i + 1;

    if inputs[oldI] == inputs[oldI + 1] {
      var oldAcc := acc;
      assert forall j :: 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j +
          ↪  1] ==> inputs[j] in oldAcc;
      assert forall d :: d in oldAcc ==> exists j :: 0 <= j < oldI && j + 1 < inputs.
          ↪ Length && inputs[j] == inputs[j + 1] && inputs[j] == d;

      acc := oldAcc + [inputs[oldI]];

      forall d | d in acc
        ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
            ↪ [j + 1] && inputs[j] == d
      {
        assert d in oldAcc + [inputs[oldI]];
        assert d in oldAcc || d in [inputs[oldI]];
        if d in oldAcc {
          var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
              ↪  inputs[j] == d;
          assert 0 <= j < nextI;
          assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
              ↪ inputs[k + 1] && inputs[k] == d;
        } else {
          assert d in [inputs[oldI]];
          assert d == inputs[oldI];
          assert 0 <= oldI < nextI;
          assert oldI + 1 < inputs.Length;
          assert inputs[oldI] == inputs[oldI + 1];
```

```
        assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] ==
            ↪ inputs[k + 1] && inputs[k] == d;
      }
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in oldAcc;
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[j] == inputs[oldI];
        assert inputs[oldI] in [inputs[oldI]];
        assert inputs[oldI] in acc;
        assert inputs[j] in acc;
      }
    }
  } else {
    forall d | d in acc
      ensures exists j :: 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs
          ↪ [j + 1] && inputs[j] == d
    {
      var j :| 0 <= j < oldI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d;
      assert 0 <= j < nextI;
      assert exists k :: 0 <= k < nextI && k + 1 < inputs.Length && inputs[k] == inputs[
          ↪ k + 1] && inputs[k] == d;
    }

    forall j | 0 <= j < nextI && j + 1 < inputs.Length && inputs[j] == inputs[j + 1]
      ensures inputs[j] in acc
    {
      if j < oldI {
        assert inputs[j] in acc;
      } else {
        assert j == oldI;
        assert inputs[oldI] != inputs[oldI + 1];
        assert inputs[j] == inputs[j + 1];
        assert false;
      }
    }
  }

  i := nextI;
}

duplicates := new int[|acc|];
var k := 0;
while k < |acc|
```

```
  invariant 0 <= k <= |acc|
  invariant duplicates.Length == |acc|
  invariant forall t :: 0 <= t < k ==> duplicates[t] == acc[t]
  decreases |acc| - k
{
  duplicates[k] := acc[k];
  k := k + 1;
}

assert duplicates.Length == |acc|;
assert |duplicates[..]| == |acc|;
forall t | 0 <= t < |acc|
  ensures duplicates[..][t] == acc[t]
{
  assert duplicates[..][t] == duplicates[t];
  assert duplicates[t] == acc[t];
}
assert duplicates[..] == acc;

assert !(i + 1 < inputs.Length);
assert i + 1 >= inputs.Length;

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in acc;
  var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
      ↪ ] == d;
  assert 0 <= j < inputs.Length;
  assert inputs[j] == d;
  assert inputs[j] in inputs[..];
}

forall j | 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates[..]
{
  assert j + 1 < inputs.Length;
  assert j + 1 < i + 1;
  assert j < i;
  assert inputs[j] in acc;
  assert inputs[j] in duplicates[..];
}

forall d | d in duplicates[..]
  ensures exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1] && inputs
      ↪ [j] == d
{
  assert d in acc;
  var j :| 0 <= j < i && j + 1 < inputs.Length && inputs[j] == inputs[j + 1] && inputs[j
      ↪ ] == d;
  assert j < inputs.Length - 1;
```

```
    assert exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[
        ↪ k] == d;
  }
}
```

Total Token Usage

Input tokens: 12389
Output tokens: 15739
Reasoning tokens: 9934
Sum of 'total tokens': 28128

Experiment Timings

Overall Experiment started at 1780653833903, ended at 1780654169144, lasting 335241ms (335.24 seconds)
Iteration #4 started at 1780654109033, ended at 1780654169143, lasting 60110ms (60.11 seconds)
Iteration #1 started at 1780653833903, ended at 1780653931544, lasting 97641ms (97.64 seconds)
Iteration #2 started at 1780653931544, ended at 1780654048206, lasting 116662ms (116.66 seconds)
Iteration #3 started at 1780654048207, ended at 1780654109033, lasting 60826ms (60.83 seconds)

Experiment 'p_4_2_d_adjacent_duplicates_seq_without_4th_ensures' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_d_adjacent_duplicates_seq_without_4th_ensures
Natural language statement: null
Method signature: p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (duplicates: seq<int>)

Ensures

- `forall d :: d in duplicates ==> d in inputs`
- `forall i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates`
- `forall d :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] && inputs[i] == d`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (duplicates
    ↪ : seq<int>)

The method should respect the following contract:

ensures forall d :: d in duplicates ==> d in inputs, ensures forall i :: 0 <= i < |inputs
    ↪ | - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates, ensures forall d
    ↪ :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i
    ↪ + 1] && inputs[i] == d

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLeb1KjrN6zprYcpxhphnY8Iqgra
Model: gpt-5.5-2026-04-23
Created at: 1780653681
Finish reason: stop
Usage: 426 tokens in, and 6414 tokens out

```
{
  duplicates := [];
  var i := 0;

  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|
    invariant forall d :: d in duplicates ==> d in inputs
    invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
        ↪ && inputs[j] == inputs[j + 1] && inputs[j] == d
    invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
        ↪  inputs[j] in duplicates
    decreases |inputs| - i
  {
    if inputs[i] == inputs[i + 1] {
      duplicates := duplicates + [inputs[i]];
    }
    i := i + 1;
```

```
  }
}
```

Final Program

```
method p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (
    ↪ duplicates: seq<int>)
       ensures forall d :: d in duplicates ==> d in inputs
       ensures forall i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] ==> inputs
           ↪ [i] in duplicates
       ensures forall d :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 &&
           ↪ inputs[i] == inputs[i + 1] && inputs[i] == d
{
  duplicates := [];
  var i := 0;

  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|
    invariant forall d :: d in duplicates ==> d in inputs
    invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
        ↪ && inputs[j] == inputs[j + 1] && inputs[j] == d
    invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
        ↪  inputs[j] in duplicates
    decreases |inputs| - i
  {
    if inputs[i] == inputs[i + 1] {
      duplicates := duplicates + [inputs[i]];
    }
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 426
Output tokens: 6414
Reasoning tokens: 6141
Sum of 'total tokens': 6840

Experiment Timings

Overall Experiment started at 1780653680997, ended at 1780653829121, lasting 148124ms (148.12 seconds)
Iteration #1 started at 1780653680998, ended at 1780653829120, lasting 148122ms (148.12 seconds)

Experiment 'p_4_3_a_extract_uppercase' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_a_extract_uppercase
Natural language statement: null
Method signature: p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)

Ensures

- `forall c :: c in uppercase ==> 'A' <= c <= 'Z'`
- `forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)

The method should respect the following contract:

ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z', ensures forall c :: c in s && 'A'
    ↪  <= c <= 'Z' ==> c in uppercase

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLmWyQr2PQMAKQzrnfDpAN7zIWDR
Model: gpt-5.5-2026-04-23
Created at: 1780654172
Finish reason: stop
Usage: 312 tokens in, and 8491 tokens out

```
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c <= 'Z'
    invariant forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in uppercase
  {
    if 'A' <= s[i] <= 'Z' {
      var previous := uppercase;
      assert forall c :: c in previous ==> 'A' <= c <= 'Z';
      assert forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in previous;
      uppercase := previous + [s[i]];

      forall c | c in uppercase
        ensures 'A' <= c <= 'Z'
      {
        assert uppercase == previous + [s[i]];
        assert exists k :: 0 <= k < |uppercase| && uppercase[k] == c;
        var k :| 0 <= k < |uppercase| && uppercase[k] == c;
```

```
      if k < |previous| {
        assert uppercase[k] == previous[k];
        assert previous[k] == c;
        assert c in previous;
        assert 'A' <= c <= 'Z';
      } else {
        assert |uppercase| == |previous| + 1;
        assert k == |previous|;
        assert k - |previous| == 0;
        assert uppercase[k] == [s[i]][k - |previous|];
        assert [s[i]][0] == s[i];
        assert c == s[i];
        assert 'A' <= c <= 'Z';
      }
    }

    forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
      ensures s[j] in uppercase
    {
      assert uppercase == previous + [s[i]];
      if j < i {
        assert s[j] in previous;
        assert s[j] in uppercase;
      } else {
        assert j == i;
        assert s[j] == s[i];
        assert s[j] in [s[i]];
        assert s[j] in uppercase;
      }
    }
  } else {
    forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
      ensures s[j] in uppercase
    {
      if j < i {
        assert s[j] in uppercase;
      } else {
        assert j == i;
        assert s[j] == s[i];
        assert 'A' <= s[i] <= 'Z';
        assert false;
      }
    }
  }
  i := i + 1;
}

forall c | c in s && 'A' <= c <= 'Z'
  ensures c in uppercase
{
  assert exists j :: 0 <= j < |s| && s[j] == c;
```

```
    var j :| 0 <= j < |s| && s[j] == c;
    assert i == |s|;
    assert 0 <= j < i;
    assert s[j] == c;
    assert 'A' <= s[j] <= 'Z';
    assert s[j] in uppercase;
    assert c in uppercase;
  }
}
```

Final Program

```
method p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)
       ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z'
       ensures forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c <= 'Z'
    invariant forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in uppercase
  {
    if 'A' <= s[i] <= 'Z' {
      var previous := uppercase;
      assert forall c :: c in previous ==> 'A' <= c <= 'Z';
      assert forall j :: 0 <= j < i && 'A' <= s[j] <= 'Z' ==> s[j] in previous;
      uppercase := previous + [s[i]];

      forall c | c in uppercase
        ensures 'A' <= c <= 'Z'
      {
        assert uppercase == previous + [s[i]];
        assert exists k :: 0 <= k < |uppercase| && uppercase[k] == c;
        var k :| 0 <= k < |uppercase| && uppercase[k] == c;
        if k < |previous| {
          assert uppercase[k] == previous[k];
          assert previous[k] == c;
          assert c in previous;
          assert 'A' <= c <= 'Z';
        } else {
          assert |uppercase| == |previous| + 1;
          assert k == |previous|;
          assert k - |previous| == 0;
          assert uppercase[k] == [s[i]][k - |previous|];
          assert [s[i]][0] == s[i];
          assert c == s[i];
          assert 'A' <= c <= 'Z';
        }
      }
```

```
    forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
      ensures s[j] in uppercase
    {
      assert uppercase == previous + [s[i]];
      if j < i {
        assert s[j] in previous;
        assert s[j] in uppercase;
      } else {
        assert j == i;
        assert s[j] == s[i];
        assert s[j] in [s[i]];
        assert s[j] in uppercase;
      }
    }
  } else {
    forall j | 0 <= j < i + 1 && 'A' <= s[j] <= 'Z'
      ensures s[j] in uppercase
    {
      if j < i {
        assert s[j] in uppercase;
      } else {
        assert j == i;
        assert s[j] == s[i];
        assert 'A' <= s[i] <= 'Z';
        assert false;
      }
    }
  }
  i := i + 1;
 }

 forall c | c in s && 'A' <= c <= 'Z'
  ensures c in uppercase
 {
  assert exists j :: 0 <= j < |s| && s[j] == c;
  var j :| 0 <= j < |s| && s[j] == c;
  assert i == |s|;
  assert 0 <= j < i;
  assert s[j] == c;
  assert 'A' <= s[j] <= 'Z';
  assert s[j] in uppercase;
  assert c in uppercase;
 }
}
```

Total Token Usage

Input tokens: 312
Output tokens: 8491

Reasoning tokens: 7610
Sum of 'total tokens': 8803

Experiment Timings

Overall Experiment started at 1780654172426, ended at 1780654353825, lasting 181399ms (181.40 seconds)
Iteration #1 started at 1780654172426, ended at 1780654353823, lasting 181397ms (181.40 seconds)

Experiment 'p_4_3_b_every_second_letter' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_b_every_second_letter
Natural language statement: null
Method signature: p_4_3_b_every_second_letter(s: string) returns (result: string)

Ensures

- `|result| * 2 == |s| || |result| * 2 + 1 == |s|`
- `forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_3_b_every_second_letter(s: string) returns (result: string)

The method should respect the following contract:

ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|, ensures forall i :: 0 <= i < |
    ↪ result| ==> result[i] == s[2 * i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLpTZCynPpxiXXhv0ssNdXMtRrh6
Model: gpt-5.5-2026-04-23
Created at: 1780654355
Finish reason: stop
Usage: 332 tokens in, and 4178 tokens out

```
{
  result := "";
  var k := 0;

  while k + 1 < |s|
    invariant 0 <= k <= |s|
    invariant k == 2 * |result|
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - k
  {
    var oldResult := result;
    var oldK := k;

    assert forall i :: 0 <= i < |oldResult| ==> oldResult[i] == s[2 * i];
    assert oldK + 1 <= |s|;

    result := oldResult + s[oldK..oldK + 1];

    assert |s[oldK..oldK + 1]| == 1;
    assert |result| == |oldResult| + 1;
```

```
    assert result[|oldResult|] == (s[oldK..oldK + 1])[0];
    assert (s[oldK..oldK + 1])[0] == s[oldK];
    assert result[|oldResult|] == s[oldK];

    assert forall i :: 0 <= i < |result| ==> result[i] == s[2 * i] by {
      forall i | 0 <= i < |result|
        ensures result[i] == s[2 * i]
      {
        if i < |oldResult| {
          assert result[i] == oldResult[i];
          assert oldResult[i] == s[2 * i];
        } else {
          assert i == |oldResult|;
          assert oldK == 2 * |oldResult|;
          assert oldK == 2 * i;
          assert result[i] == s[oldK];
        }
      }
    }

    k := oldK + 2;
    assert k == 2 * |result|;
    assert k <= |s|;
  }

  assert |s| <= k + 1;
  if k == |s| {
    assert |result| * 2 == |s|;
  } else {
    assert k < |s|;
    assert k + 1 == |s|;
    assert |result| * 2 + 1 == |s|;
  }
}
```

Final Program

```
method p_4_3_b_every_second_letter(s: string) returns (result: string)
       ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|
       ensures forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
{
  result := "";
  var k := 0;

  while k + 1 < |s|
    invariant 0 <= k <= |s|
    invariant k == 2 * |result|
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - k
  {
```

```
    var oldResult := result;
    var oldK := k;

    assert forall i :: 0 <= i < |oldResult| ==> oldResult[i] == s[2 * i];
    assert oldK + 1 <= |s|;

    result := oldResult + s[oldK..oldK + 1];

    assert |s[oldK..oldK + 1]| == 1;
    assert |result| == |oldResult| + 1;
    assert result[|oldResult|] == (s[oldK..oldK + 1])[0];
    assert (s[oldK..oldK + 1])[0] == s[oldK];
    assert result[|oldResult|] == s[oldK];

    assert forall i :: 0 <= i < |result| ==> result[i] == s[2 * i] by {
      forall i | 0 <= i < |result|
        ensures result[i] == s[2 * i]
      {
        if i < |oldResult| {
          assert result[i] == oldResult[i];
          assert oldResult[i] == s[2 * i];
        } else {
          assert i == |oldResult|;
          assert oldK == 2 * |oldResult|;
          assert oldK == 2 * i;
          assert result[i] == s[oldK];
        }
      }
    }

    k := oldK + 2;
    assert k == 2 * |result|;
    assert k <= |s|;
  }

  assert |s| <= k + 1;
  if k == |s| {
    assert |result| * 2 == |s|;
  } else {
    assert k < |s|;
    assert k + 1 == |s|;
    assert |result| * 2 + 1 == |s|;
  }
}
```

Total Token Usage

Input tokens: 332
Output tokens: 4178
Reasoning tokens: 3583

Sum of 'total tokens': 4510

Experiment Timings

Overall Experiment started at 1780654355180, ended at 1780654449655, lasting 94475ms (94.48 seconds)
Iteration #1 started at 1780654355181, ended at 1780654449655, lasting 94474ms (94.47 seconds)

Experiment 'p_4_3_c_replace_vowels' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_3_c_replace_vowels
Natural language statement: null
Method signature: p_4_3_c_replace_vowels(s: string) returns (result: string)

Ensures

- `|result| == |s|`
- `forall i :: 0 <= i < |s| ==> (if s[i] in ['a','e','i','o','u','A','E','I','O','U'] then result[i] == '_' else result[i] == s[i])`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_3_c_replace_vowels(s: string) returns (result: string)

The method should respect the following contract:

ensures |result| == |s|, ensures forall i :: 0 <= i < |s| ==> (if s[i] in ['a','e','i','o
    ↪ ','u','A','E','I','O','U'] then result[i] == '_' else result[i] == s[i])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLr2ePkjCYio3vBviVLX8JUvb1La
Model: gpt-5.5-2026-04-23
Created at: 1780654452
Finish reason: stop
Usage: 360 tokens in, and 1599 tokens out

```
{
  result := seq(|s|, i => if s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']
      ↪ then '_' else s[i]);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_3_c_replace_vowels/program.dfy(5,30): Error: index out of range
  |
5 | result := seq(|s|, i => if s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']
    ↪  then '_' else s[i]);
  | ^


Dafny program verifier finished with 1 verified, 1 error
```

```
Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLrptA3ZIs53lDNJFJLcHpq6bzpW
Model: gpt-5.5-2026-04-23
Created at: 1780654501
Finish reason: stop
Usage: 612 tokens in, and 710 tokens out

```
{
  result := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant |result| == i
    invariant forall j :: 0 <= j < i ==> (if s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', '
        ↪ I', 'O', 'U'] then result[j] == '_' else result[j] == s[j])
  {
    var c := if s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'] then '_' else s
        ↪ [i];
    result := result + [c];
    i := i + 1;
  }
}
```

Final Program

```
method p_4_3_c_replace_vowels(s: string) returns (result: string)
       ensures |result| == |s|
       ensures forall i :: 0 <= i < |s| ==> (if s[i] in ['a','e','i','o','u','A','E','I
           ↪ ','O','U'] then result[i] == '_' else result[i] == s[i])
{
  result := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant |result| == i
    invariant forall j :: 0 <= j < i ==> (if s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', '
        ↪ I', 'O', 'U'] then result[j] == '_' else result[j] == s[j])
  {
    var c := if s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'] then '_' else s
        ↪ [i];
    result := result + [c];
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 972
Output tokens: 2309
Reasoning tokens: 2026
Sum of 'total tokens': 3281

Experiment Timings

Overall Experiment started at 1780654451849, ended at 1780654533683, lasting 81834ms (81.83 seconds)
Iteration #1 started at 1780654451849, ended at 1780654500858, lasting 49009ms (49.01 seconds)
Iteration #2 started at 1780654500859, ended at 1780654533683, lasting 32824ms (32.82 seconds)

Experiment 'p_4_3_d_count_vowels' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_d_count_vowels
Natural language statement: null
Method signature: p_4_3_d_count_vowels(s: string) returns (count: int)

Ensures

- `count == count_vowels(s)`

Functional Code Given

```
function count_vowels(s: string): int {
  if s == "" then 0
  else if s[0] in ['a','e','i','o','u','A','E','I','O','U'] then count_vowels(s[1..]) + 1
  else count_vowels(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_3_d_count_vowels(s: string) returns (count: int)

The method should respect the following contract:

ensures count == count_vowels(s)

The contract uses the following dafny code:

function count_vowels(s: string): int {
  if s == "" then 0
  else if s[0] in ['a','e','i','o','u','A','E','I','O','U'] then count_vowels(s[1..]) + 1
  else count_vowels(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLsPEaCc8puF001rzt9bD03sRAaN
Model: gpt-5.5-2026-04-23
Created at: 1780654537
Finish reason: stop
Usage: 444 tokens in, and 1991 tokens out

```
{
  if s == "" {
    count := 0;
  } else {
    var rest := p_4_3_d_count_vowels(s[1..]);
    if s[0] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'] {
      count := rest + 1;
    } else {
      count := rest;
    }
  }
```

```
}
```

Final Program

```
function count_vowels(s: string): int {
  if s == "" then 0
  else if s[0] in ['a','e','i','o','u','A','E','I','O','U'] then count_vowels(s[1..]) + 1
  else count_vowels(s[1..])
}

method p_4_3_d_count_vowels(s: string) returns (count: int)
       ensures count == count_vowels(s)
{
  if s == "" {
    count := 0;
  } else {
    var rest := p_4_3_d_count_vowels(s[1..]);
    if s[0] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'] {
      count := rest + 1;
    } else {
      count := rest;
    }
  }
}
```

Total Token Usage

Input tokens: 444
Output tokens: 1991
Reasoning tokens: 1847
Sum of 'total tokens': 2435

Experiment Timings

Overall Experiment started at 1780654536650, ended at 1780654605840, lasting 69190ms (69.19 seconds)
Iteration #1 started at 1780654536669, ended at 1780654605840, lasting 69171ms (69.17 seconds)

Experiment 'p_4_3_e_positions_of_vowels_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_3_e_positions_of_vowels_array
Natural language statement: null
Method signature: p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)

Ensures

- `forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U`
- `forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions[..]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)

The method should respect the following contract:

ensures forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','
    ↪ A','E','I','O','U'], ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o
    ↪ ','u','A','E','I','O','U'] ==> i in positions[..]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLwJihORMBWU7lzhooM67NgmmuLA
Model: gpt-5.5-2026-04-23
Created at: 1780654779
Finish reason: stop
Usage: 418 tokens in, and 5552 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in acc ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A','E
        ↪ ','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in acc
  {
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      acc := acc + [i];
    }
    assert forall p :: p in acc ==> 0 <= p < i + 1 && s[p] in ['a','e','i','o','u','A','E
        ↪ ','I','O','U'];
    assert forall j :: 0 <= j < i + 1 && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪  ==> j in acc;
    i := i + 1;
  }
```

```
  positions := new int[|acc|](j => acc[j]);
  assert positions.Length == |acc|;
  assert forall j :: 0 <= j < |acc| ==> positions[j] == acc[j];
  assert forall j :: 0 <= j < |positions[..]| ==> positions[..][j] == acc[j];
  assert positions[..] == acc;
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_3_e_positions_of_vowels_array/program.dfy(20,38): Error: index out of
    ↪ range
   |
20 | positions := new int[|acc|](j => acc[j]);
   |                              ^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLyWamrzHEJ92q28iVzhLhZpjmey
Model: gpt-5.5-2026-04-23
Created at: 1780654916
Finish reason: stop
Usage: 878 tokens in, and 2449 tokens out

```
{
  var acc: seq<int> := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in acc ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A','E
        ↪ ','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in acc
  {
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      acc := acc + [i];
    }
    assert forall p :: p in acc ==> 0 <= p < i + 1 && s[p] in ['a','e','i','o','u','A','E
        ↪ ','I','O','U'];
    assert forall j :: 0 <= j < i + 1 && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪  ==> j in acc;
```

```
    i := i + 1;
  }

  positions := new int[|acc|](j => if 0 <= j < |acc| then acc[j] else 0);
  assert positions.Length == |acc|;
  assert forall j :: 0 <= j < |acc| ==> positions[j] == acc[j];
  assert forall j :: 0 <= j < |positions[..]| ==> positions[..][j] == acc[j];
  assert positions[..] == acc;
}
```

Final Program

```
method p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)
       ensures forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o
           ↪ ','u','A','E','I','O','U']
       ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','
           ↪ U'] ==> i in positions[..]
{
  var acc: seq<int> := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in acc ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A','E
        ↪ ','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in acc
  {
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      acc := acc + [i];
    }
    assert forall p :: p in acc ==> 0 <= p < i + 1 && s[p] in ['a','e','i','o','u','A','E
        ↪ ','I','O','U'];
    assert forall j :: 0 <= j < i + 1 && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪  ==> j in acc;
    i := i + 1;
  }

  positions := new int[|acc|](j => if 0 <= j < |acc| then acc[j] else 0);
  assert positions.Length == |acc|;
  assert forall j :: 0 <= j < |acc| ==> positions[j] == acc[j];
  assert forall j :: 0 <= j < |positions[..]| ==> positions[..][j] == acc[j];
  assert positions[..] == acc;
}
```

Total Token Usage

Input tokens: 1296
Output tokens: 8001

Reasoning tokens: 7161
Sum of 'total tokens': 9297

Experiment Timings

Overall Experiment started at 1780654777109, ended at 1780654992354, lasting 215245ms (215.25 seconds)
Iteration #1 started at 1780654777110, ended at 1780654915953, lasting 138843ms (138.84 seconds)
Iteration #2 started at 1780654915953, ended at 1780654992354, lasting 76401ms (76.40 seconds)

Experiment 'p_4_3_e_positions_of_vowels_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_e_positions_of_vowels_seq
Natural language statement: null
Method signature: p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)

Ensures

- `forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U']`
- `forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)

The method should respect the following contract:

ensures forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','
    ↪ E','I','O','U'], ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u
    ↪ ','A','E','I','O','U'] ==> i in positions

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLtXhqxaeIR5emJysII9hjmF5SYu
Model: gpt-5.5-2026-04-23
Created at: 1780654607
Finish reason: stop
Usage: 412 tokens in, and 6119 tokens out

```
{
  positions := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in positions ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A
        ↪ ','E','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in positions
  {
    var next := i + 1;
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      var previous := positions;
      assert forall p :: p in previous ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A
          ↪ ','E','I','O','U'];
      assert forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
          ↪ ==> j in previous;

      positions := previous + [i];
```

```
  assert forall p :: p in positions ==> p in previous || p == i;
  assert forall p :: p in previous ==> p in positions;
  assert i in positions;

  assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u
      ↪ ','A','E','I','O','U'] by {
    forall p | p in positions
      ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
    {
      if p == i {
        assert 0 <= i < next;
        assert s[p] == s[i];
      } else {
        assert p in previous;
        assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
        assert p < next;
      }
    }
  }

  assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U
      ↪ '] ==> j in positions by {
    forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
      ensures j in positions
    {
      if j == i {
        assert i in positions;
      } else {
        assert j < i;
        assert j in previous;
        assert j in positions;
      }
    }
  }
} else {
  assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u
      ↪ ','A','E','I','O','U'] by {
    forall p | p in positions
      ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
    {
      assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
      assert p < next;
    }
  }

  assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U
      ↪ '] ==> j in positions by {
    forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
      ensures j in positions
    {
```

```
        if j == i {
          assert s[j] == s[i];
          assert s[i] in ['a','e','i','o','u','A','E','I','O','U'];
          assert !(s[i] in ['a','e','i','o','u','A','E','I','O','U']);
          assert false;
        } else {
          assert j < i;
          assert j in positions;
        }
       }
      }
    }
    i := next;
  }
}
```

Final Program

```
method p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)
      ensures forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u
          ↪ ','A','E','I','O','U']
      ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','
          ↪ U'] ==> i in positions
{
  positions := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in positions ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A
        ↪ ','E','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in positions
  {
    var next := i + 1;
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      var previous := positions;
      assert forall p :: p in previous ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A
          ↪ ','E','I','O','U'];
      assert forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
          ↪ ==> j in previous;

      positions := previous + [i];

      assert forall p :: p in positions ==> p in previous || p == i;
      assert forall p :: p in previous ==> p in positions;
      assert i in positions;

      assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u
          ↪ ','A','E','I','O','U'] by {
        forall p | p in positions
```

```
      ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
    {
      if p == i {
        assert 0 <= i < next;
        assert s[p] == s[i];
      } else {
        assert p in previous;
        assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
        assert p < next;
      }
    }
  }

  assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U
      ↪ '] ==> j in positions by {
    forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
      ensures j in positions
    {
      if j == i {
        assert i in positions;
      } else {
        assert j < i;
        assert j in previous;
        assert j in positions;
      }
    }
  }
} else {
  assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u
      ↪ ','A','E','I','O','U'] by {
    forall p | p in positions
      ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
    {
      assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
      assert p < next;
    }
  }

  assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U
      ↪ '] ==> j in positions by {
    forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
      ensures j in positions
    {
      if j == i {
        assert s[j] == s[i];
        assert s[i] in ['a','e','i','o','u','A','E','I','O','U'];
        assert !(s[i] in ['a','e','i','o','u','A','E','I','O','U']);
        assert false;
      } else {
        assert j < i;
        assert j in positions;
```

```
        }
      }
    }
  }
  i := next;
 }
}
```

Total Token Usage

Input tokens: 412
Output tokens: 6119
Reasoning tokens: 5057
Sum of 'total tokens': 6531

Experiment Timings

Overall Experiment started at 1780654606900, ended at 1780654772341, lasting 165441ms (165.44 seconds)
Iteration #1 started at 1780654606903, ended at 1780654772341, lasting 165438ms (165.44 seconds)

Experiment 'p_4_4_month_with_highest_temperature_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_4_month_with_highest_temperature_array
Natural language statement: null
Method signature: p_4_4_month_with_highest_temperature_array(inputs: array<real>) returns (hottest: int)

Ensures

- `1 <= hottest <= 12`
- `forall i :: 0 <= i <= inputs.Length - 1 ==> inputs[hottest - 1] >= inputs[i]`

Requires

- `inputs.Length == 12`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_4_month_with_highest_temperature_array(inputs: array<real>) returns (hottest: int)

The method should respect the following contract:

requires inputs.Length == 12, ensures 1 <= hottest <= 12, ensures forall i :: 0 <= i <=
    ↪ inputs.Length - 1 ==> inputs[hottest - 1] >= inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnM1YA1MBz2akYAJ0XZuEyOTcqm0u
Model: gpt-5.5-2026-04-23
Created at: 1780655104
Finish reason: stop
Usage: 326 tokens in, and 6717 tokens out

```
{
  hottest := 1;
  var idx := 1;

  forall i | 0 <= i < idx
    ensures inputs[hottest - 1] >= inputs[i]
  {
    assert i == 0;
    assert hottest - 1 == 0;
    assert inputs[hottest - 1] == inputs[i];
  }

  while idx < inputs.Length
    invariant 1 <= hottest <= inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant forall i :: 0 <= i < idx ==> inputs[hottest - 1] >= inputs[i]
    decreases inputs.Length - idx
  {
    if inputs[idx] > inputs[hottest - 1] {
      var previousHottest := hottest;
```

```
      assert inputs[idx] > inputs[previousHottest - 1];
      assert forall i :: 0 <= i < idx ==> inputs[previousHottest - 1] >= inputs[i];

      hottest := idx + 1;
      assert hottest - 1 == idx;
      assert inputs[hottest - 1] == inputs[idx];
      assert inputs[hottest - 1] >= inputs[idx];

      forall i | 0 <= i < idx
        ensures inputs[hottest - 1] >= inputs[i]
      {
        assert inputs[previousHottest - 1] >= inputs[i];
        assert inputs[idx] > inputs[previousHottest - 1];
        assert inputs[idx] >= inputs[i];
        assert inputs[hottest - 1] == inputs[idx];
      }
    } else {
      assert !(inputs[idx] > inputs[hottest - 1]);
      assert inputs[idx] <= inputs[hottest - 1];
      assert inputs[hottest - 1] >= inputs[idx];
    }

    assert inputs[hottest - 1] >= inputs[idx];
    assert forall i :: 0 <= i < idx ==> inputs[hottest - 1] >= inputs[i];

    forall i | 0 <= i < idx + 1
      ensures inputs[hottest - 1] >= inputs[i]
    {
      if i < idx {
        assert inputs[hottest - 1] >= inputs[i];
      } else {
        assert i == idx;
        assert inputs[hottest - 1] >= inputs[idx];
      }
    }

    idx := idx + 1;
  }

  assert idx == inputs.Length;
  assert inputs.Length == 12;
  assert 1 <= hottest <= 12;

  forall i | 0 <= i <= inputs.Length - 1
    ensures inputs[hottest - 1] >= inputs[i]
  {
    assert i < inputs.Length;
    assert i < idx;
    assert inputs[hottest - 1] >= inputs[i];
  }
}
```

Final Program

```
method p_4_4_month_with_highest_temperature_array(inputs: array<real>) returns (hottest:
    ↪ int)
       requires inputs.Length == 12
       ensures 1 <= hottest <= 12
       ensures forall i :: 0 <= i <= inputs.Length - 1 ==> inputs[hottest - 1] >= inputs[
           ↪ i]
{
  hottest := 1;
  var idx := 1;

  forall i | 0 <= i < idx
    ensures inputs[hottest - 1] >= inputs[i]
  {
    assert i == 0;
    assert hottest - 1 == 0;
    assert inputs[hottest - 1] == inputs[i];
  }

  while idx < inputs.Length
    invariant 1 <= hottest <= inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant forall i :: 0 <= i < idx ==> inputs[hottest - 1] >= inputs[i]
    decreases inputs.Length - idx
  {
    if inputs[idx] > inputs[hottest - 1] {
      var previousHottest := hottest;
      assert inputs[idx] > inputs[previousHottest - 1];
      assert forall i :: 0 <= i < idx ==> inputs[previousHottest - 1] >= inputs[i];

      hottest := idx + 1;
      assert hottest - 1 == idx;
      assert inputs[hottest - 1] == inputs[idx];
      assert inputs[hottest - 1] >= inputs[idx];

      forall i | 0 <= i < idx
        ensures inputs[hottest - 1] >= inputs[i]
      {
        assert inputs[previousHottest - 1] >= inputs[i];
        assert inputs[idx] > inputs[previousHottest - 1];
        assert inputs[idx] >= inputs[i];
        assert inputs[hottest - 1] == inputs[idx];
      }
    } else {
      assert !(inputs[idx] > inputs[hottest - 1]);
      assert inputs[idx] <= inputs[hottest - 1];
      assert inputs[hottest - 1] >= inputs[idx];
    }

    assert inputs[hottest - 1] >= inputs[idx];
    assert forall i :: 0 <= i < idx ==> inputs[hottest - 1] >= inputs[i];
```

```
    forall i | 0 <= i < idx + 1
      ensures inputs[hottest - 1] >= inputs[i]
    {
      if i < idx {
        assert inputs[hottest - 1] >= inputs[i];
      } else {
        assert i == idx;
        assert inputs[hottest - 1] >= inputs[idx];
      }
    }

    idx := idx + 1;
  }

  assert idx == inputs.Length;
  assert inputs.Length == 12;
  assert 1 <= hottest <= 12;

  forall i | 0 <= i <= inputs.Length - 1
    ensures inputs[hottest - 1] >= inputs[i]
  {
    assert i < inputs.Length;
    assert i < idx;
    assert inputs[hottest - 1] >= inputs[i];
  }
}
```

Total Token Usage

Input tokens: 326
Output tokens: 6717
Reasoning tokens: 5987
Sum of 'total tokens': 7043

Experiment Timings

Overall Experiment started at 1780655102802, ended at 1780655277410, lasting 174608ms (174.61 seconds)
Iteration #1 started at 1780655102805, ended at 1780655277410, lasting 174605ms (174.61 seconds)

Experiment 'p_4_4_month_with_highest_temperature_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_4_month_with_highest_temperature_seq
Natural language statement: null
Method signature: p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)

Ensures

- `1 <= hottest <= 12`
- `forall i :: 0 <= i <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]`

Requires

- `|inputs| == 12`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)

The method should respect the following contract:

requires |inputs| == 12, ensures 1 <= hottest <= 12, ensures forall i :: 0 <= i <= |
    ↪ inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnLzqHA5hn1ezF7eNRCRyhTNDCCO2
Model: gpt-5.5-2026-04-23
Created at: 1780654998
Finish reason: stop
Usage: 330 tokens in, and 4110 tokens out

```
{
  var maxIdx: int := 0;
  var i: int := 1;

  while i < |inputs|
    invariant |inputs| == 12
    invariant 1 <= i <= |inputs|
    invariant 0 <= maxIdx < i
    invariant forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] > inputs[maxIdx] {
      var prev := maxIdx;
      assert inputs[i] > inputs[prev];
      assert forall j :: 0 <= j < i ==> inputs[prev] >= inputs[j];

      maxIdx := i;
      assert inputs[maxIdx] >= inputs[i];

      assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j] by {
```

```
      forall j | 0 <= j < i
        ensures inputs[maxIdx] >= inputs[j]
      {
        assert inputs[prev] >= inputs[j];
        assert inputs[maxIdx] == inputs[i];
        assert inputs[i] > inputs[prev];
      }
    }
  } else {
    assert inputs[maxIdx] >= inputs[i];
    assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
  }

  assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
  assert forall j :: 0 <= j < i + 1 ==> inputs[maxIdx] >= inputs[j] by {
    forall j | 0 <= j < i + 1
      ensures inputs[maxIdx] >= inputs[j]
    {
      if j < i {
        assert inputs[maxIdx] >= inputs[j];
      } else {
        assert j == i;
        assert inputs[maxIdx] >= inputs[i];
      }
    }
  }

  i := i + 1;
 }

 hottest := maxIdx + 1;
 assert i == |inputs|;
 assert 0 <= maxIdx < |inputs|;
 assert |inputs| == 12;
 assert maxIdx < 12;
 assert maxIdx <= 11;
 assert 1 <= hottest <= 12;
 assert hottest - 1 == maxIdx;

 assert forall j :: 0 <= j <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[j] by {
   forall j | 0 <= j <= |inputs| - 1
     ensures inputs[hottest - 1] >= inputs[j]
   {
     assert j < |inputs|;
     assert inputs[maxIdx] >= inputs[j];
     assert hottest - 1 == maxIdx;
   }
 }
}
```

Final Program

```
method p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)
       requires |inputs| == 12
       ensures 1 <= hottest <= 12
       ensures forall i :: 0 <= i <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]
{
  var maxIdx: int := 0;
  var i: int := 1;

  while i < |inputs|
    invariant |inputs| == 12
    invariant 1 <= i <= |inputs|
    invariant 0 <= maxIdx < i
    invariant forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] > inputs[maxIdx] {
      var prev := maxIdx;
      assert inputs[i] > inputs[prev];
      assert forall j :: 0 <= j < i ==> inputs[prev] >= inputs[j];

      maxIdx := i;
      assert inputs[maxIdx] >= inputs[i];

      assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j] by {
        forall j | 0 <= j < i
          ensures inputs[maxIdx] >= inputs[j]
        {
          assert inputs[prev] >= inputs[j];
          assert inputs[maxIdx] == inputs[i];
          assert inputs[i] > inputs[prev];
        }
      }
    } else {
      assert inputs[maxIdx] >= inputs[i];
      assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
    }

    assert forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j];
    assert forall j :: 0 <= j < i + 1 ==> inputs[maxIdx] >= inputs[j] by {
      forall j | 0 <= j < i + 1
        ensures inputs[maxIdx] >= inputs[j]
      {
        if j < i {
          assert inputs[maxIdx] >= inputs[j];
        } else {
          assert j == i;
          assert inputs[maxIdx] >= inputs[i];
        }
      }
    }
```

```
    i := i + 1;
  }

  hottest := maxIdx + 1;
  assert i == |inputs|;
  assert 0 <= maxIdx < |inputs|;
  assert |inputs| == 12;
  assert maxIdx < 12;
  assert maxIdx <= 11;
  assert 1 <= hottest <= 12;
  assert hottest - 1 == maxIdx;

  assert forall j :: 0 <= j <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[j] by {
    forall j | 0 <= j <= |inputs| - 1
      ensures inputs[hottest - 1] >= inputs[j]
    {
      assert j < |inputs|;
      assert inputs[maxIdx] >= inputs[j];
      assert hottest - 1 == maxIdx;
    }
  }
}
```

Total Token Usage

Input tokens: 330
Output tokens: 4110
Reasoning tokens: 3427
Sum of 'total tokens': 4440

Experiment Timings

Overall Experiment started at 1780654995350, ended at 1780655099881, lasting 104531ms (104.53 seconds)
Iteration #1 started at 1780654995350, ended at 1780655099880, lasting 104530ms (104.53 seconds)

Experiment 'p_4_5_analyse_floats_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 2
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_5_analyse_floats_array
Natural language statement: null
Method signature: p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values[..]) / values.Length as real)`
- `forall v :: v in values[..] ==> min <= v`
- `forall v :: v in values[..] ==> max >= v`
- `range == max - min`
- `exists v :: v in values[..] && min == v`
- `exists v :: v in values[..] && max == v`

Requires

- `0 < values.Length`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max:
    ↪ real, range: real)

The method should respect the following contract:

requires 0 < values.Length, ensures average == (sum(values[..]) / values.Length as real),
    ↪  ensures forall v :: v in values[..] ==> min <= v, ensures forall v :: v in values
    ↪ [..] ==> max >= v, ensures range == max - min, ensures exists v :: v in values[..]
    ↪  && min == v, ensures exists v :: v in values[..] && max == v

The contract uses the following dafny code:

function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnM9Bw34NxU9eIMyDa18zxnByCahV
Model: gpt-5.5-2026-04-23
Created at: 1780655577
Finish reason: stop
Usage: 558 tokens in, and 8302 tokens out

```
{
  var n := values.Length;

  var total := 0.0;
  var i := n;
  while 0 < i
    invariant 0 <= i <= n
```

```
  invariant total == sum(values[i..n])
{
  i := i - 1;
  assert 0 <= i < n;
  assert total == sum(values[i + 1..n]);
  assert |values[i..n]| > 0;
  assert (values[i..n])[0] == values[i];
  assert (values[i..n])[1..] == values[i + 1..n];
  assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
  total := values[i] + total;
  assert total == sum(values[i..n]);
}
assert i == 0;
assert values[0..n] == values[..];
assert total == sum(values[..]);

min := values[0];
max := values[0];
var minIndex := 0;
var maxIndex := 0;
i := 1;
while i < n
  invariant 1 <= i <= n
  invariant forall k: int :: 0 <= k < i ==> min <= values[k]
  invariant forall k: int :: 0 <= k < i ==> max >= values[k]
  invariant 0 <= minIndex < i
  invariant min == values[minIndex]
  invariant 0 <= maxIndex < i
  invariant max == values[maxIndex]
{
  var x := values[i];
  if x < min {
    min := x;
    minIndex := i;
  }
  if x > max {
    max := x;
    maxIndex := i;
  }
  i := i + 1;
}
assert i == n;
assert forall k: int :: 0 <= k < n ==> min <= values[k];
assert forall k: int :: 0 <= k < n ==> max >= values[k];

average := total / (n as real);
range := max - min;
assert n == values.Length;
assert average == sum(values[..]) / (values.Length as real);

assert forall v: real :: v in values[..] ==> min <= v by {
```

```
    if v in values[..] {
      assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
      var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
      assert |values[..]| == n;
      assert values[..][k] == values[k];
      assert v == values[k];
      assert min <= values[k];
    }
  }
  assert forall v: real :: v in values[..] ==> max >= v by {
    if v in values[..] {
      assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
      var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
      assert |values[..]| == n;
      assert values[..][k] == values[k];
      assert v == values[k];
      assert max >= values[k];
    }
  }

  assert 0 <= minIndex < |values[..]|;
  assert values[..][minIndex] == min;
  assert min in values[..];
  assert exists v: real :: v in values[..] && min == v;

  assert 0 <= maxIndex < |values[..]|;
  assert values[..][maxIndex] == max;
  assert max in values[..];
  assert exists v: real :: v in values[..] && max == v;
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_5_analyse_floats_array/program.dfy(73,7): Error: unresolved identifier:
    ↪ v
   |
73 | if v in values[..] {
   | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(74,74): Error: unresolved identifier:
    ↪  v
   |
74 | assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
   | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(75,64): Error: unresolved identifier:
    ↪  v
   |
75 | var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
```

```
   | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(78,13): Error: unresolved identifier:
    ↪ v
   |
78 | assert v == values[k];
   | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(83,7): Error: unresolved identifier:
    ↪ v
   |
83 | if v in values[..] {
   | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(84,74): Error: unresolved identifier:
    ↪ v
   |
84 | assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
   | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(85,64): Error: unresolved identifier:
    ↪ v
   |
85 | var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
   | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(88,13): Error: unresolved identifier:
    ↪ v
   |
88 | assert v == values[k];
   | ^

8 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnMBykXOfm2DvAaqbWCCKlezP1kMg
Model: gpt-5.5-2026-04-23
Created at: 1780655750
Finish reason: stop
Usage: 2226 tokens in, and 1894 tokens out

```
{
  var n := values.Length;

  var total := 0.0;
```

```
var i := n;
while 0 < i
  invariant 0 <= i <= n
  invariant total == sum(values[i..n])
{
  i := i - 1;
  assert 0 <= i < n;
  assert total == sum(values[i + 1..n]);
  assert |values[i..n]| > 0;
  assert (values[i..n])[0] == values[i];
  assert (values[i..n])[1..] == values[i + 1..n];
  assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
  total := values[i] + total;
  assert total == sum(values[i..n]);
}
assert i == 0;
assert values[0..n] == values[..];
assert total == sum(values[..]);

min := values[0];
max := values[0];
var minIndex := 0;
var maxIndex := 0;
i := 1;
while i < n
  invariant 1 <= i <= n
  invariant forall k: int :: 0 <= k < i ==> min <= values[k]
  invariant forall k: int :: 0 <= k < i ==> max >= values[k]
  invariant 0 <= minIndex < i
  invariant min == values[minIndex]
  invariant 0 <= maxIndex < i
  invariant max == values[maxIndex]
{
  var x := values[i];
  if x < min {
    min := x;
    minIndex := i;
  }
  if x > max {
    max := x;
    maxIndex := i;
  }
  i := i + 1;
}
assert i == n;
assert forall k: int :: 0 <= k < n ==> min <= values[k];
assert forall k: int :: 0 <= k < n ==> max >= values[k];

average := total / (n as real);
range := max - min;
assert n == values.Length;
```

```
  assert average == sum(values[..]) / (values.Length as real);

  assert forall v: real | v in values[..]
    ensures min <= v
  {
    assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
    var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
    assert |values[..]| == n;
    assert (values[..])[k] == values[k];
    assert v == values[k];
    assert min <= values[k];
  }
  assert forall v: real | v in values[..]
    ensures max >= v
  {
    assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
    var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
    assert |values[..]| == n;
    assert (values[..])[k] == values[k];
    assert v == values[k];
    assert max >= values[k];
  }

  assert 0 <= minIndex < |values[..]|;
  assert (values[..])[minIndex] == min;
  assert min in values[..];
  assert exists v: real :: v in values[..] && min == v;

  assert 0 <= maxIndex < |values[..]|;
  assert (values[..])[maxIndex] == max;
  assert max in values[..];
  assert exists v: real :: v in values[..] && max == v;
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_5_analyse_floats_array/program.dfy(102,0): Error: invalid
    ↪ UnaryExpression
    |
102 | }
    | ^

problem_data/p_4_5_analyse_floats_array/program.dfy(101,54): Error: expected either 'by'
    ↪ or a semicolon following the assert expression
    |
101 | assert exists v: real :: v in values[..] && max == v;
    | ^
```

```
problem_data/p_4_5_analyse_floats_array/program.dfy(101,54): Error: expected either 'by'
    ↪ or a semicolon following the assert expression
    |
101 | assert exists v: real :: v in values[..] && max == v;
    | ^

3 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnMCiRHOL2DrFiPseTRFDQk5jMVN1
Model: gpt-5.5-2026-04-23
Created at: 1780655796
Finish reason: stop
Usage: 3094 tokens in, and 2444 tokens out

```
{
  var n := values.Length;

  var total := 0.0;
  var i := n;
  while 0 < i
    invariant 0 <= i <= n
    invariant total == sum(values[i..n])
  {
    i := i - 1;
    assert 0 <= i < n;
    assert total == sum(values[i + 1..n]);
    assert |values[i..n]| > 0;
    assert (values[i..n])[0] == values[i];
    assert (values[i..n])[1..] == values[i + 1..n];
    assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
    total := values[i] + total;
    assert total == sum(values[i..n]);
  }
  assert i == 0;
  assert values[0..n] == values[..];
  assert total == sum(values[..]);

  min := values[0];
  max := values[0];
  var minIndex := 0;
  var maxIndex := 0;
  i := 1;
  while i < n
    invariant 1 <= i <= n
```

```
  invariant forall k: int :: 0 <= k < i ==> min <= values[k]
  invariant forall k: int :: 0 <= k < i ==> max >= values[k]
  invariant 0 <= minIndex < i
  invariant min == values[minIndex]
  invariant 0 <= maxIndex < i
  invariant max == values[maxIndex]
{
  var x := values[i];
  if x < min {
    min := x;
    minIndex := i;
  }
  if x > max {
    max := x;
    maxIndex := i;
  }
  i := i + 1;
}
assert i == n;
assert forall k: int :: 0 <= k < n ==> min <= values[k];
assert forall k: int :: 0 <= k < n ==> max >= values[k];

average := total / (n as real);
range := max - min;
assert n == values.Length;
assert average == sum(values[..]) / (values.Length as real);

forall v: real | v in values[..]
  ensures min <= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert min <= values[k];
}
forall v: real | v in values[..]
  ensures max >= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert max >= values[k];
}

assert 0 <= minIndex < |values[..]|;
assert (values[..])[minIndex] == min;
assert min in values[..];
```

```
  assert exists v: real :: (v in values[..]) && min == v;

  assert 0 <= maxIndex < |values[..]|;
  assert (values[..])[maxIndex] == max;
  assert max in values[..];
  assert exists v: real :: (v in values[..]) && max == v;
}
```

Final Program

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real,
    ↪ max: real, range: real)
       requires 0 < values.Length
       ensures average == (sum(values[..]) / values.Length as real)
       ensures forall v :: v in values[..] ==> min <= v
       ensures forall v :: v in values[..] ==> max >= v
       ensures range == max - min
       ensures exists v :: v in values[..] && min == v
       ensures exists v :: v in values[..] && max == v
{
  var n := values.Length;

  var total := 0.0;
  var i := n;
  while 0 < i
    invariant 0 <= i <= n
    invariant total == sum(values[i..n])
  {
    i := i - 1;
    assert 0 <= i < n;
    assert total == sum(values[i + 1..n]);
    assert |values[i..n]| > 0;
    assert (values[i..n])[0] == values[i];
    assert (values[i..n])[1..] == values[i + 1..n];
    assert sum(values[i..n]) == values[i] + sum(values[i + 1..n]);
    total := values[i] + total;
    assert total == sum(values[i..n]);
  }
  assert i == 0;
  assert values[0..n] == values[..];
  assert total == sum(values[..]);

  min := values[0];
  max := values[0];
```

```
var minIndex := 0;
var maxIndex := 0;
i := 1;
while i < n
  invariant 1 <= i <= n
  invariant forall k: int :: 0 <= k < i ==> min <= values[k]
  invariant forall k: int :: 0 <= k < i ==> max >= values[k]
  invariant 0 <= minIndex < i
  invariant min == values[minIndex]
  invariant 0 <= maxIndex < i
  invariant max == values[maxIndex]
{
  var x := values[i];
  if x < min {
    min := x;
    minIndex := i;
  }
  if x > max {
    max := x;
    maxIndex := i;
  }
  i := i + 1;
}
assert i == n;
assert forall k: int :: 0 <= k < n ==> min <= values[k];
assert forall k: int :: 0 <= k < n ==> max >= values[k];

average := total / (n as real);
range := max - min;
assert n == values.Length;
assert average == sum(values[..]) / (values.Length as real);

forall v: real | v in values[..]
  ensures min <= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert min <= values[k];
}
forall v: real | v in values[..]
  ensures max >= v
{
  assert exists k: int :: 0 <= k < |values[..]| && (values[..])[k] == v;
  var k: int :| 0 <= k < |values[..]| && (values[..])[k] == v;
  assert |values[..]| == n;
  assert (values[..])[k] == values[k];
  assert v == values[k];
  assert max >= values[k];
```

```
  }

  assert 0 <= minIndex < |values[..]|;
  assert (values[..])[minIndex] == min;
  assert min in values[..];
  assert exists v: real :: (v in values[..]) && min == v;

  assert 0 <= maxIndex < |values[..]|;
  assert (values[..])[maxIndex] == max;
  assert max in values[..];
  assert exists v: real :: (v in values[..]) && max == v;
}
```

Total Token Usage

Input tokens: 5878
Output tokens: 12640
Reasoning tokens: 9834
Sum of 'total tokens': 18518

Experiment Timings

Overall Experiment started at 1780655576606, ended at 1780655867699, lasting 291093ms (291.09 seconds)
Iteration #1 started at 1780655576606, ended at 1780655749995, lasting 173389ms (173.39 seconds)
Iteration #2 started at 1780655749996, ended at 1780655796534, lasting 46538ms (46.54 seconds)
Iteration #3 started at 1780655796534, ended at 1780655867699, lasting 71165ms (71.17 seconds)

Experiment 'p_4_5_analyse_floats_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_5_analyse_floats_seq
Natural language statement: null
Method signature: p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values) / |values| as real)`
- `forall v :: v in values ==> min <= v`
- `forall v :: v in values ==> max >= v`
- `range == max - min`
- `exists v :: v in values && min == v`
- `exists v :: v in values && max == v`

Requires

- `0 < |values|`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real,
    ↪  range: real)

The method should respect the following contract:

requires 0 < |values|, ensures average == (sum(values) / |values| as real), ensures
    ↪ forall v :: v in values ==> min <= v, ensures forall v :: v in values ==> max >= v
    ↪ , ensures range == max - min, ensures exists v :: v in values && min == v, ensures
    ↪  exists v :: v in values && max == v

The contract uses the following dafny code:

function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnM4R5NQEDMncFxjzNj3Bjo6WLjT8
Model: gpt-5.5-2026-04-23
Created at: 1780655283
Finish reason: stop
Usage: 544 tokens in, and 8453 tokens out

```
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := 0.0;
```

```
  assert values[0] in values;
  assert values[1..] == [];

  forall v: real | v in values
    ensures v == values[0]
  {
    var k: int :| 0 <= k < |values| && values[k] == v;
    assert k == 0;
  }

  assert exists v: real :: v in values && min == v;
  assert exists v: real :: v in values && max == v;
} else {
  assert 1 < |values|;
  assert 0 < |values[1..]|;

  var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(values[1..]);
  var tailLen: real := |values[1..]| as real;
  assert tailLen != 0.0;
  assert |values| == 1 + |values[1..]|;

  average := (values[0] + tailAverage * tailLen) / (|values| as real);

  var headIsMin := values[0] <= tailMin;
  if headIsMin {
    min := values[0];
  } else {
    min := tailMin;
  }

  var headIsMax := values[0] >= tailMax;
  if headIsMax {
    max := values[0];
  } else {
    max := tailMax;
  }

  range := max - min;

  assert values[0] in values;

  forall v: real | v in values[1..]
    ensures v in values
  {
    var k: int :| 0 <= k < |values[1..]| && values[1..][k] == v;
    assert 0 <= k + 1 < |values|;
    assert values[1..][k] == values[k + 1];
    assert values[k + 1] == v;
    assert v in values;
  }
```

```
forall v: real | v in values
  ensures v == values[0] || v in values[1..]
{
  var k: int :| 0 <= k < |values| && values[k] == v;
  if k == 0 {
    assert v == values[0];
  } else {
    assert 0 <= k - 1 < |values[1..]|;
    assert values[1..][k - 1] == values[k];
    assert values[1..][k - 1] == v;
    assert v in values[1..];
  }
}

forall v: real | v in values
  ensures min <= v
{
  if v == values[0] {
    if headIsMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert !headIsMin;
    }
  } else {
    assert v in values[1..];
    if headIsMin {
      assert min == values[0];
      assert values[0] <= tailMin;
      assert tailMin <= v;
    } else {
      assert min == tailMin;
      assert tailMin <= v;
    }
  }
}

forall v: real | v in values
  ensures max >= v
{
  if v == values[0] {
    if headIsMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert !headIsMax;
    }
  } else {
    assert v in values[1..];
    if headIsMax {
      assert max == values[0];
```

```
        assert values[0] >= tailMax;
        assert tailMax >= v;
      } else {
        assert max == tailMax;
        assert tailMax >= v;
      }
    }
  }

  if headIsMin {
    assert min == values[0];
    assert values[0] in values && min == values[0];
    assert exists v: real :: v in values && min == v;
  } else {
    assert min == tailMin;
    var w: real :| w in values[1..] && tailMin == w;
    assert w in values;
    assert w in values && min == w;
    assert exists v: real :: v in values && min == v;
  }

  if headIsMax {
    assert max == values[0];
    assert values[0] in values && max == values[0];
    assert exists v: real :: v in values && max == v;
  } else {
    assert max == tailMax;
    var w: real :| w in values[1..] && tailMax == w;
    assert w in values;
    assert w in values && max == w;
    assert exists v: real :: v in values && max == v;
  }
 }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_5_analyse_floats_seq/program.dfy(15,0): Error: a postcondition could not
    ↪ be proved on this return path
   |
15 | {
   | ^

problem_data/p_4_5_analyse_floats_seq/program.dfy(9,17): Related location: this is the
    ↪ postcondition that could not be proved
  |
9 |  ensures average == (sum(values) / |values| as real)
  |  ^^
```

```
Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnM7womErwQeXiTlXfm9hnd3JfqTI
Model: gpt-5.5-2026-04-23
Created at: 1780655500
Finish reason: stop
Usage: 1900 tokens in, and 3101 tokens out

```
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := 0.0;

    assert values[0] in values;
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);

    forall v: real | v in values
      ensures v == values[0]
    {
      var k: int :| 0 <= k < |values| && values[k] == v;
      assert k == 0;
    }

    assert exists v: real :: v in values && min == v;
    assert exists v: real :: v in values && max == v;
  } else {
    assert 1 < |values|;
    assert 0 < |values[1..]|;

    var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(values[1..]);
    var tailLen: real := |values[1..]| as real;
    var len: real := |values| as real;
    assert tailLen > 0.0;
    assert tailLen != 0.0;
    assert |values| == 1 + |values[1..]|;
    assert len == 1.0 + tailLen;
```

```
assert tailAverage == sum(values[1..]) / tailLen;
assert (sum(values[1..]) / tailLen) * tailLen == sum(values[1..]);
assert tailAverage * tailLen == sum(values[1..]);

average := (values[0] + tailAverage * tailLen) / len;

assert sum(values) == values[0] + sum(values[1..]);
calc {
  average;
  ==
  (values[0] + tailAverage * tailLen) / len;
  ==
  (values[0] + sum(values[1..])) / len;
  ==
  sum(values) / len;
  ==
  sum(values) / (|values| as real);
}

var headIsMin := values[0] <= tailMin;
if headIsMin {
  min := values[0];
} else {
  min := tailMin;
}

var headIsMax := values[0] >= tailMax;
if headIsMax {
  max := values[0];
} else {
  max := tailMax;
}

range := max - min;

assert values[0] in values;

forall v: real | v in values[1..]
  ensures v in values
{
  var k: int :| 0 <= k < |values[1..]| && values[1..][k] == v;
  assert 0 <= k + 1 < |values|;
  assert values[1..][k] == values[k + 1];
  assert values[k + 1] == v;
  assert v in values;
}

forall v: real | v in values
  ensures v == values[0] || v in values[1..]
{
```

```
  var k: int :| 0 <= k < |values| && values[k] == v;
  if k == 0 {
    assert v == values[0];
  } else {
    assert 0 <= k - 1 < |values[1..]|;
    assert values[1..][k - 1] == values[k];
    assert values[1..][k - 1] == v;
    assert v in values[1..];
  }
}

forall v: real | v in values
  ensures min <= v
{
  if v == values[0] {
    if headIsMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert !(values[0] <= tailMin);
      assert tailMin < values[0];
      assert tailMin <= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMin {
      assert min == values[0];
      assert values[0] <= tailMin;
      assert tailMin <= v;
    } else {
      assert min == tailMin;
      assert tailMin <= v;
    }
  }
}

forall v: real | v in values
  ensures max >= v
{
  if v == values[0] {
    if headIsMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert !(values[0] >= tailMax);
      assert values[0] < tailMax;
      assert tailMax >= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMax {
```

```
        assert max == values[0];
        assert values[0] >= tailMax;
        assert tailMax >= v;
      } else {
        assert max == tailMax;
        assert tailMax >= v;
      }
    }
  }

  if headIsMin {
    assert min == values[0];
    assert values[0] in values && min == values[0];
    assert exists v: real :: v in values && min == v;
  } else {
    assert min == tailMin;
    var w: real :| w in values[1..] && tailMin == w;
    assert w in values;
    assert w in values && min == w;
    assert exists v: real :: v in values && min == v;
  }

  if headIsMax {
    assert max == values[0];
    assert values[0] in values && max == values[0];
    assert exists v: real :: v in values && max == v;
  } else {
    assert max == tailMax;
    var w: real :| w in values[1..] && tailMax == w;
    assert w in values;
    assert w in values && max == w;
    assert exists v: real :: v in values && max == v;
  }
 }
}
```

Final Program

```
function sum(values: seq<real>): real
{
 if |values| == 0 then 0.0
 else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max
    ↪ : real, range: real)
      requires 0 < |values|
      ensures average == (sum(values) / |values| as real)
      ensures forall v :: v in values ==> min <= v
      ensures forall v :: v in values ==> max >= v
```

```
      ensures range == max - min
      ensures exists v :: v in values && min == v
      ensures exists v :: v in values && max == v
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := 0.0;

    assert values[0] in values;
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);

    forall v: real | v in values
      ensures v == values[0]
    {
      var k: int :| 0 <= k < |values| && values[k] == v;
      assert k == 0;
    }

    assert exists v: real :: v in values && min == v;
    assert exists v: real :: v in values && max == v;
  } else {
    assert 1 < |values|;
    assert 0 < |values[1..]|;

    var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(values[1..]);
    var tailLen: real := |values[1..]| as real;
    var len: real := |values| as real;
    assert tailLen > 0.0;
    assert tailLen != 0.0;
    assert |values| == 1 + |values[1..]|;
    assert len == 1.0 + tailLen;

    assert tailAverage == sum(values[1..]) / tailLen;
    assert (sum(values[1..]) / tailLen) * tailLen == sum(values[1..]);
    assert tailAverage * tailLen == sum(values[1..]);

    average := (values[0] + tailAverage * tailLen) / len;

    assert sum(values) == values[0] + sum(values[1..]);
    calc {
      average;
      ==
      (values[0] + tailAverage * tailLen) / len;
      ==
```

```
  (values[0] + sum(values[1..])) / len;
  ==
  sum(values) / len;
  ==
  sum(values) / (|values| as real);
}

var headIsMin := values[0] <= tailMin;
if headIsMin {
  min := values[0];
} else {
  min := tailMin;
}

var headIsMax := values[0] >= tailMax;
if headIsMax {
  max := values[0];
} else {
  max := tailMax;
}

range := max - min;

assert values[0] in values;

forall v: real | v in values[1..]
  ensures v in values
{
  var k: int :| 0 <= k < |values[1..]| && values[1..][k] == v;
  assert 0 <= k + 1 < |values|;
  assert values[1..][k] == values[k + 1];
  assert values[k + 1] == v;
  assert v in values;
}

forall v: real | v in values
  ensures v == values[0] || v in values[1..]
{
  var k: int :| 0 <= k < |values| && values[k] == v;
  if k == 0 {
    assert v == values[0];
  } else {
    assert 0 <= k - 1 < |values[1..]|;
    assert values[1..][k - 1] == values[k];
    assert values[1..][k - 1] == v;
    assert v in values[1..];
  }
}

forall v: real | v in values
  ensures min <= v
```

```
{
  if v == values[0] {
    if headIsMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert !(values[0] <= tailMin);
      assert tailMin < values[0];
      assert tailMin <= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMin {
      assert min == values[0];
      assert values[0] <= tailMin;
      assert tailMin <= v;
    } else {
      assert min == tailMin;
      assert tailMin <= v;
    }
  }
}

forall v: real | v in values
  ensures max >= v
{
  if v == values[0] {
    if headIsMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert !(values[0] >= tailMax);
      assert values[0] < tailMax;
      assert tailMax >= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMax {
      assert max == values[0];
      assert values[0] >= tailMax;
      assert tailMax >= v;
    } else {
      assert max == tailMax;
      assert tailMax >= v;
    }
  }
}

if headIsMin {
  assert min == values[0];
  assert values[0] in values && min == values[0];
```

```
    assert exists v: real :: v in values && min == v;
   } else {
    assert min == tailMin;
    var w: real :| w in values[1..] && tailMin == w;
    assert w in values;
    assert w in values && min == w;
    assert exists v: real :: v in values && min == v;
   }

   if headIsMax {
    assert max == values[0];
    assert values[0] in values && max == values[0];
    assert exists v: real :: v in values && max == v;
   } else {
    assert max == tailMax;
    var w: real :| w in values[1..] && tailMax == w;
    assert w in values;
    assert w in values && max == w;
    assert exists v: real :: v in values && max == v;
   }
  }
}
```

Total Token Usage

Input tokens: 2444
Output tokens: 11554
Reasoning tokens: 8677
Sum of 'total tokens': 13998

Experiment Timings

Overall Experiment started at 1780655281597, ended at 1780655574700, lasting 293103ms (293.10 seconds)
Iteration #1 started at 1780655281664, ended at 1780655499280, lasting 217616ms (217.62 seconds)
Iteration #2 started at 1780655499280, ended at 1780655574700, lasting 75420ms (75.42 seconds)

Experiment 'p_4_6_find_minimum_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_6_find_minimum_array
Natural language statement: null
Method signature: p_4_6_find_minimum_array(inputs: array<int>) returns (min: int)

Ensures

- `forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]`
- `exists i :: 0 <= i < inputs.Length && min == inputs[i]`

Requires

- `0 < inputs.Length`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_6_find_minimum_array(inputs: array<int>) returns (min: int)

The method should respect the following contract:

requires 0 < inputs.Length, ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[
    ↪ i], ensures exists i :: 0 <= i < inputs.Length && min == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnMEU4XDheo3TJfVAVMcEjKv8LOom
Model: gpt-5.5-2026-04-23
Created at: 1780655906
Finish reason: stop
Usage: 316 tokens in, and 566 tokens out

```
{
  min := inputs[0];
  var idx := 1;
  while idx < inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant forall i :: 0 <= i < idx ==> min <= inputs[i]
    invariant exists i :: 0 <= i < idx && min == inputs[i]
  {
    if inputs[idx] < min {
      min := inputs[idx];
    }
    idx := idx + 1;
  }
}
```

Final Program

```
method p_4_6_find_minimum_array(inputs: array<int>) returns (min: int)
```

```
      requires 0 < inputs.Length
      ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]
      ensures exists i :: 0 <= i < inputs.Length && min == inputs[i]
{
  min := inputs[0];
  var idx := 1;
  while idx < inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant forall i :: 0 <= i < idx ==> min <= inputs[i]
    invariant exists i :: 0 <= i < idx && min == inputs[i]
  {
    if inputs[idx] < min {
      min := inputs[idx];
    }
    idx := idx + 1;
  }
}
```

Total Token Usage

Input tokens: 316
Output tokens: 566
Reasoning tokens: 441
Sum of 'total tokens': 882

Experiment Timings

Overall Experiment started at 1780655906131, ended at 1780655935184, lasting 29053ms (29.05 seconds)
Iteration #1 started at 1780655906133, ended at 1780655935184, lasting 29051ms (29.05 seconds)

Experiment 'p_4_6_find_minimum_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_6_find_minimum_seq
Natural language statement: null
Method signature: p_4_6_find_minimum_seq(inputs: seq<int>) returns (min: int)

Ensures

- `forall i :: 0 <= i < |inputs| ==> min <= inputs[i]`
- `exists i :: 0 <= i < |inputs| && min == inputs[i]`

Requires

- `0 < |inputs|`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_6_find_minimum_seq(inputs: seq<int>) returns (min: int)

The method should respect the following contract:

requires 0 < |inputs|, ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i],
    ↪ ensures exists i :: 0 <= i < |inputs| && min == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnMDuoa62x4jvmrDq3n8GCona37NY
Model: gpt-5.5-2026-04-23
Created at: 1780655870
Finish reason: stop
Usage: 322 tokens in, and 1023 tokens out

```
{
  var idx := 1;
  var minIndex := 0;
  min := inputs[0];

  while idx < |inputs|
    invariant 1 <= idx <= |inputs|
    invariant 0 <= minIndex < idx
    invariant min == inputs[minIndex]
    invariant forall i :: 0 <= i < idx ==> min <= inputs[i]
  {
    if inputs[idx] < min {
      min := inputs[idx];
      minIndex := idx;
    }
    idx := idx + 1;
  }
}
```

Final Program

```
method p_4_6_find_minimum_seq(inputs: seq<int>) returns (min: int)
       requires 0 < |inputs|
       ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
       ensures exists i :: 0 <= i < |inputs| && min == inputs[i]
{
  var idx := 1;
  var minIndex := 0;
  min := inputs[0];

  while idx < |inputs|
    invariant 1 <= idx <= |inputs|
    invariant 0 <= minIndex < idx
    invariant min == inputs[minIndex]
    invariant forall i :: 0 <= i < idx ==> min <= inputs[i]
  {
    if inputs[idx] < min {
      min := inputs[idx];
      minIndex := idx;
    }
    idx := idx + 1;
  }
}
```

Total Token Usage

Input tokens: 322
Output tokens: 1023
Reasoning tokens: 871
Sum of 'total tokens': 1345

Experiment Timings

Overall Experiment started at 1780655869708, ended at 1780655904330, lasting 34622ms (34.62 seconds)
Iteration #1 started at 1780655869710, ended at 1780655904330, lasting 34620ms (34.62 seconds)

Experiment 'p_4_8_seperate_lines' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_8_seperate_lines
Natural language statement: null
Method signature: p_4_8_seperate_lines(s: string) returns (chars: string)

Ensures

- `|chars| == 2 * |s|`
- `forall i :: 0 <= i < |s| ==> i % 2 == 1 ==> chars[i] == '\n'`
- `forall i :: 0 <= i < |s| ==> i % 2 == 0 ==> chars[i] == s[i / 2]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_8_seperate_lines(s: string) returns (chars: string)

The method should respect the following contract:

ensures |chars| == 2 * |s|, ensures forall i :: 0 <= i < |s| ==> i % 2 == 1 ==> chars[i]
    ↪ == '\n', ensures forall i :: 0 <= i < |s| ==> i % 2 == 0 ==> chars[i] == s[i / 2]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnMFRk4H1ZDLDY637i98uUCbqmKl7
Model: gpt-5.5-2026-04-23
Created at: 1780655965
Finish reason: stop
Usage: 374 tokens in, and 1099 tokens out

```
{
  chars := seq(2 * |s|, i requires 0 <= i < 2 * |s| =>
    if i % 2 == 0 then s[i / 2] else '\n');
}
```

Final Program

```
method p_4_8_seperate_lines(s: string) returns (chars: string)
       ensures |chars| == 2 * |s|
       ensures forall i :: 0 <= i < |s| ==> i % 2 == 1 ==> chars[i] == '\n'
       ensures forall i :: 0 <= i < |s| ==> i % 2 == 0 ==> chars[i] == s[i / 2]
{
  chars := seq(2 * |s|, i requires 0 <= i < 2 * |s| =>
    if i % 2 == 0 then s[i / 2] else '\n');
}
```

Total Token Usage

Input tokens: 374
Output tokens: 1099
Reasoning tokens: 1024
Sum of 'total tokens': 1473

Experiment Timings

Overall Experiment started at 1780655965283, ended at 1780656004710, lasting 39427ms (39.43 seconds)
Iteration #1 started at 1780655965286, ended at 1780656004710, lasting 39424ms (39.42 seconds)

Experiment 'p_4_9_reverse_string' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_9_reverse_string
Natural language statement: null
Method signature: p_4_9_reverse_string(s: string) returns (reversed: string)

Ensures

- `|reversed| == |s|`
- `forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_9_reverse_string(s: string) returns (reversed: string)

The method should respect the following contract:

ensures |reversed| == |s|, ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1
    ↪ - i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnMG6Y79InhlaJbP6ZA03lXLMbptV
Model: gpt-5.5-2026-04-23
Created at: 1780656006
Finish reason: stop
Usage: 308 tokens in, and 3802 tokens out

```
{
  var n := |s|;
  reversed := "";
  var i := n;

  while i > 0
    invariant n == |s|
    invariant 0 <= i <= n
    invariant |reversed| == n - i
    invariant forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k]
    decreases i
  {
    var oldReversed := reversed;
    var oldI := i;
    assert |oldReversed| == n - oldI;
    assert forall k :: 0 <= k < |oldReversed| ==> oldReversed[k] == s[n - 1 - k];

    i := i - 1;
    assert oldI == i + 1;
    assert 0 <= i < n;
```

```
    var piece := s[i..i+1];
    assert |piece| == 1;
    assert piece[0] == s[i];

    reversed := oldReversed + piece;
    assert |reversed| == |oldReversed| + |piece|;
    assert |reversed| == n - i;

    assert forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k] by {
      forall k | 0 <= k < |reversed|
        ensures reversed[k] == s[n - 1 - k]
      {
        if k < |oldReversed| {
          assert reversed[k] == oldReversed[k];
          assert oldReversed[k] == s[n - 1 - k];
        } else {
          assert k >= |oldReversed|;
          assert k < |oldReversed| + |piece|;
          assert |piece| == 1;
          assert k == |oldReversed|;
          assert reversed[k] == piece[0];
          assert piece[0] == s[i];
          assert |oldReversed| == n - oldI;
          assert oldI == i + 1;
          assert |oldReversed| == n - (i + 1);
          assert k == n - (i + 1);
          assert n - 1 - k == i;
          assert reversed[k] == s[n - 1 - k];
        }
      }
    }
  }

  assert |reversed| == |s|;
  assert forall k :: 0 <= k < |s| ==> reversed[k] == s[|s| - 1 - k] by {
    forall k | 0 <= k < |s|
      ensures reversed[k] == s[|s| - 1 - k]
    {
      assert k < |reversed|;
      assert n == |s|;
      assert reversed[k] == s[n - 1 - k];
      assert n - 1 - k == |s| - 1 - k;
    }
  }
}
```

Final Program

```
method p_4_9_reverse_string(s: string) returns (reversed: string)
```

```
      ensures |reversed| == |s|
      ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]
{
  var n := |s|;
  reversed := "";
  var i := n;

  while i > 0
    invariant n == |s|
    invariant 0 <= i <= n
    invariant |reversed| == n - i
    invariant forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k]
    decreases i
  {
    var oldReversed := reversed;
    var oldI := i;
    assert |oldReversed| == n - oldI;
    assert forall k :: 0 <= k < |oldReversed| ==> oldReversed[k] == s[n - 1 - k];

    i := i - 1;
    assert oldI == i + 1;
    assert 0 <= i < n;

    var piece := s[i..i+1];
    assert |piece| == 1;
    assert piece[0] == s[i];

    reversed := oldReversed + piece;
    assert |reversed| == |oldReversed| + |piece|;
    assert |reversed| == n - i;

    assert forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k] by {
      forall k | 0 <= k < |reversed|
        ensures reversed[k] == s[n - 1 - k]
      {
        if k < |oldReversed| {
          assert reversed[k] == oldReversed[k];
          assert oldReversed[k] == s[n - 1 - k];
        } else {
          assert k >= |oldReversed|;
          assert k < |oldReversed| + |piece|;
          assert |piece| == 1;
          assert k == |oldReversed|;
          assert reversed[k] == piece[0];
          assert piece[0] == s[i];
          assert |oldReversed| == n - oldI;
          assert oldI == i + 1;
          assert |oldReversed| == n - (i + 1);
          assert k == n - (i + 1);
          assert n - 1 - k == i;
          assert reversed[k] == s[n - 1 - k];
```

```
      }
    }
  }
 }

 assert |reversed| == |s|;
 assert forall k :: 0 <= k < |s| ==> reversed[k] == s[|s| - 1 - k] by {
  forall k | 0 <= k < |s|
    ensures reversed[k] == s[|s| - 1 - k]
  {
    assert k < |reversed|;
    assert n == |s|;
    assert reversed[k] == s[n - 1 - k];
    assert n - 1 - k == |s| - 1 - k;
  }
 }
}
```

Total Token Usage

Input tokens: 308
Output tokens: 3802
Reasoning tokens: 3072
Sum of 'total tokens': 4110

Experiment Timings

Overall Experiment started at 1780656006396, ended at 1780656102777, lasting 96381ms (96.38 seconds)
Iteration #1 started at 1780656006396, ended at 1780656102777, lasting 96381ms (96.38 seconds)